uc3m | Universidad Carlos III de Madrid

Bachelor's Degree in Data Science and Engineering

Academic Year 2024-2025

Data Science Project - Final Report

"Anomaly detection and prediction using clinical sensors data streams"

Mónica De Álvaro Mena

Paula Gutiérrez Arroyo

Diego Hernández Suárez

Alejandro Taboada Esteban

Tutor: Marcelino Lázaro

January 2025

This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

This project is being carried out in collaboration with the Pediatric University Hospital Niño Jesús, leveraging data collected from sensors within the Intensive Care Unit (ICU). The focus of the project is the development and implementation of an anomaly detection system based on time series analysis, designed to identify irregular patterns that may signal potential threats among patients.

The approach involves the application of a combination of advanced machine learning algorithms and statistical methods to accurately detect anomalies in real-time, thereby enabling the anticipation of high-risk situations. This process will be enhanced through the use of continuous data monitoring.

By identifying deviations in vital signs or other time-dependent monitored parameters, the system aims to support medical staff in making prompt decisions, ultimately optimizing patient outcomes. This initiative has the potential to improve clinical monitoring protocols and increase the efficiency of responses to emergent threats within the ICU environment.

Keywords: Anomaly Detection, Outlier Detection, Clinical Monitoring, Statistical Methods, Machine Learning, Supervised Learning, Unsupervised Learning, Deep Learning, Time Series, Data Monitoring, AutoRegressive Integrated Moving Average (ARIMA), Isolation Forest, One-class Support Vector Machines, Sliding Window Technique, Neural Networks, K-Means, Clustering, Long Short-Term Memory (LSTM), Autoencoders

Contents

1 Introduction **1**

1.1 Overview of the Project 1

1.2 Project Objectives 1

1.3 Document Content Description 2

2 State of the art **3**

2.1 Anomaly Detection Methods 3

2.1.1 Statistical Methods 3

2.1.2 Machine Learning-Based Approaches 4

2.1.3 Time Series Anomaly Detection 8

2.1.4 Anomaly Detection with Healthcare Data 9

2.2 Change Point Detection 10

2.2.1 Statistical Methods 10

2.2.2 Machine Learning Methods 13

3 Data analysis **15**

3.1 Problems with availability of the data 15

3.2 Description of the dataset VitalDB 15

3.3 Challenges with the Dataset 16

3.4 Time representation and Frequency 17

3.5 Focusing on other signals, based on the Hospital's feedback 17

4 Data preprocessing **19**

5 Anomaly detection **21**

5.1 One-Class SVM model implementation 21

5.2 Isolation forest model implementation 23
5.3 LSTM model implementation 25
5.4 Autoencoders 27
5.4.1 Dense 28
5.4.2 Convolutional Neural Network 30
5.4.3 Transformer 31

6 Change Point Detection **34**
6.1 Statistical Methods 34
6.1.1 Parametric Models 34
6.1.2 Non-Parametric Models 35
6.2 Ruptures library 38
6.2.1 Bottom-Up CPD 38
6.2.2 Other Algorithms employed 40
6.3 Machine Learning Models 47
6.3.1 Clustering 47
6.3.2 LSTM-based methods 51

7 Challenges and conclusions **57**
7.1 Challenges and Limitations 57
7.2 Final Conclusions 58

List of Acronyms

AI Artificial Intelligence

ARIMA AutoRegressive Integrated Moving Average

CNN Convolutional Neural Network

CPD Change Point Detection

CUSUM Cumulative Sum

DBSCAN Density-Based Spatial Clustering of Applications with Noise

ECG Electrocardiogram

EEG Electroencephalogram

HMM Hidden Markov Model

ZS Z-scores

MSE Mean Squared Error

MRI Magnetic Resonance Imaging

GAN Generative Adversarial Network

GMM Gaussian Mixture Model

ICU Intensive Care Units

IQR Interquartile Range

IoT Internet of Things

KDDCup Knowledge Discovery and Data Mining Cup

LSTM Long Short-Term Memory

NN Neural Networks

RBF Radial Basis Function

OCSVM One-Class Support Vector Machine

RNN Recurrent Neural Network

SARIMA Seasonal AutoRegressive Integrated Moving Average

SVM Support Vector Machine

WD Window Delta

GB Gradient Boosting

XGBoost Extreme Gradient Boosting

Chapter 1

Introduction

1.1 Overview of the Project

In Intensive Care Units (ICU) at hospitals like the Pediatric University Hospital Niño Jesús, patients are continuously monitored through sensors that collect real-time data on their vital signs. When properly analyzed, this data can help differentiate true medical emergencies from anomalies that trigger false alarms. In this context, there is a need for an advanced anomaly detection system capable of filtering out false alarms while accurately identifying genuine threats to patient health, allowing for rapid and effective medical intervention.

This project aims to develop and implement an anomaly detection system alongside a change point detection mechanism, leveraging time series analysis. By integrating machine learning algorithms with statistical methods, the objective is to create a real-time system capable of reducing false alarms and accurately identifying medical emergencies. Additionally, the system will monitor for significant changes in patients' conditions. The ultimate goal is to enhance clinical outcomes by distinguishing between normal fluctuations and critical abnormalities in vital signs or other monitored parameters, enabling the early detection of genuine risks.

1.2 Project Objectives

1. **Develop** a system that detects anomalies in clinical data through time series analysis, while also identifying significant change points in patients' conditions.

2. **Integrate** statistical methods and advanced machine learning techniques to identify irregular patterns and detect critical transitions in clinical data.

3. **Propose** an adaptable solution that can scale with the continuously generated data in the clinical environment, ensuring real-time performance and reliabil-

ity.

1.3 Document Content Description

This document outlines the development and implementation of an anomaly detection and change point detection system for ICU data, combining statistical and machine learning approaches. It begins with an introduction to the problem, project objectives, and the motivations behind using advanced analytical methods in clinical settings. A review of the state of the art provides an overview of current techniques for anomaly and change point detection, highlighting their applicability to time series data and healthcare contexts.

The following sections delve into the dataset used, focusing on its characteristics, challenges in data availability, and the preprocessing steps required to prepare it for analysis. Methods for anomaly detection are then presented, covering models such as One-Class SVM, Isolation Forest, LSTMs, and Autoencoders. Change point detection is explored in detail, including statistical methods like parametric and non-parametric methods, as well as machine learning-based techniques, with discussions on their implementation and challenges encountered during the process.

Finally, the document concludes with a presentation of results, evaluating the performance of the proposed system and discussing its implications for real-world clinical applications. Recommendations for future work and potential areas for improvement are also outlined, emphasizing the importance of such systems in enhancing patient monitoring and outcomes in ICU environments.

Chapter 2

State of the art

Anomaly detection, also referred to as outlier detection, plays a critical role in a wide range of fields, though our main focus will be in its applications in healthcare. The core idea is to identify patterns or events that deviate significantly from the expected behavior of a system, often signaling rare, critical, or unexpected events. Over time, numerous techniques have emerged, leveraging both statistical and machine learning approaches to refine and improve anomaly detection.

2.1 Anomaly Detection Methods

Anomaly detection using machine learning plays a critical role in identifying irregular patterns within datasets [1, 2], especially in fields like medical signal processing where precision is vital [2]. In essence, anomaly detection refers to the task of identifying rare or unusual events that deviate from the norm. These anomalies may indicate critical conditions such as abnormal heart rhythms, breathing irregularities, or signs of neurological disorders in medical signals like Electrocardiogram (ECG), Electroencephalogram (EEG), or Magnetic Resonance Imaging (MRI) data.

2.1.1 Statistical Methods

Traditional anomaly detection techniques were rooted in statistical models. These approaches, like Z-scores, Interquartile Range (IQR), AutoRegressive Integrated Moving Average (ARIMA), and parametric modeling (e.g., Gaussian Mixture Model (GMM)), assume that anomalies are deviations from a normal distribution. However, such methods can be limited when the underlying distribution is complex or unknown [3]. Nevertheless, using techniques like ARIMA models, provide a simple way, that can still prove effective, to detect anomalies. By predicting the time series with the model, we can detect if a point is an anomaly if it deviates above a certain threshold from the prediction.

Z-scores and IQR are useful when the data follows a relatively normal distribution, helping to identify outliers by examining how far a data point deviates from the mean. For instance, data points with Z-scores greater than a certain value (e.g., 3 standard deviations from the mean) can be flagged as anomalies. Similarly, the IQR method identifies outliers by looking at the spread of the middle 50% of the data, flagging any points that fall outside 1.5 times the IQR.

Parametric models, such as GMMs, assume that the data is generated from a mixture of several normal distributions. These models estimate the parameters of each distribution and calculate the likelihood of each data point belonging to a particular distribution. Anomalies are then detected as points with low probability under the estimated model, as they do not fit any of the normal distributions well.

In time series analysis, ARIMA models are particularly useful for capturing temporal dependencies and trends in the data. The model uses past values (autoregressive), differencing (to make the data stationary), and moving averages to forecast future values. Anomalies are identified when the observed data deviates significantly from the model's predictions. Specifically, a threshold is set, and any point that lies beyond this threshold is flagged as an anomaly. ARIMA models are particularly effective when the time series exhibits clear seasonal or trending behavior.

Despite their simplicity, statistical methods remain popular due to their interpretability and efficiency. However, they often struggle in high-dimensional data or when the assumptions of normality and independence do not hold. In such cases, more advanced machine learning or deep learning methods are often required to capture complex, non-linear relationships and dependencies in the data.

2.1.2 Machine Learning-Based Approaches

In recent years, machine learning techniques have gained prominence due to their ability to detect more complex and non-linear patterns. Particularly, unsupervised and semi-supervised models are highly effective in detecting anomalies without requiring large labeled datasets. Algorithms such as clustering, autoencoders [4], One-Class Support Vector Machine (OCSVM) [5], or Isolation Forests [6] can learn from historical data and detect subtle changes that may signal anomaly conditions.

Supervised Learning

In cases where **labeled data** is available, supervised learning algorithms such as Support Vector Machine (SVM), Decision Trees, Extreme Gradient Boosting (XGBoost) and Neural Networks (NN) are commonly employed for anomaly detection. The key idea behind these methods is to create a model that learns from historical data containing both normal and anomalous instances, allowing it to classify new, unseen

data points.

SVMs [7] are a popular choice for supervised anomaly detection due to their ability to handle high-dimensional data. SVM constructs a hyperplane in a multi-dimensional space that separates data points of different classes (normal vs. anomalous). The goal of the SVM is to find the optimal hyperplane that maximizes the margin between the classes, and for anomaly detection, it distinguishes between majority (normal) and minority (anomalous) behavior, even when anomalies are rare [5].

Decision Trees are another effective supervised learning method for anomaly detection. They recursively partition the feature space into subsets, building a tree where each leaf represents a class (normal or anomaly). The tree is trained using **labeled data**, and during inference, new data points traverse the tree to reach a leaf node that classifies the point. Decision Trees offer interpretable models and can handle non-linear relationships between features, and are the foundation of more complex models like Random Forests [8].

XGBoost is a highly efficient machine learning technique that has shown great success in various competitions, including Kaggle and Knowledge Discovery and Data Mining Cup (KDDCup). XGBoost operates by building a sequence of decision trees, each trained to correct the errors of its predecessor. One of its main strengths is its scalability, making it well-suited for large datasets. In the context of anomaly detection, XGBoost is typically used for time-series forecasting, where anomalies are identified based on the prediction error. If a prediction deviates beyond a predefined threshold from the actual observation, it is flagged as an anomaly [3].

NN [9] are powerful tools for detecting anomalies in high-dimensional datasets. Consisting of interconnected layers of neurons, these networks learn to approximate a function mapping input data to classification labels. During training, weights are adjusted by minimizing a loss function through algorithms like stochastic gradient descent. Once trained, the network identifies anomalies based on learned patterns. Although NN have been used in many applications for years [10], the advent of deep learning [11] significantly revitalized interest in these networks. Deep learning's ability to handle complex, large-scale data problems, combined with computational advances and algorithmic improvements, reignited interest in neural networks and made them the dominant paradigm in AI research and industry applications today. For a deeper discussion of deep learning techniques, refer to the next subsection Deep Learning.

The common theme in supervised learning is that the models rely on **labeled datasets** where the distinction between normal and anomalous data is explicitly provided. This allows models to capture patterns and characteristics that distinguish the two, enabling classification of new instances. However, obtaining sufficient **labeled data** for both classes, especially for anomalies, can be challenging in real-

world anomaly detection tasks due to the rarity of anomalous events.

Unsupervised Learning

In most real-world scenarios, labeled anomalous data is scarce, making unsupervised techniques particularly relevant for anomaly detection. Unlike supervised methods, unsupervised learning techniques do not require labeled data and instead rely on the intrinsic structure of the data to identify anomalies. Clustering methods like K-Means, Density-Based Spatial Clustering of Applications with Noise (DBSCAN), and algorithms like Isolation Forest or OCSVM are commonly used to detect anomalies based on deviations from dense clusters of data points.

Isolation Forest [6] is a tree-based algorithm specifically designed for anomaly detection. The idea behind Isolation Forest is that anomalies are few and different from normal points, making them easier to isolate. The algorithm randomly selects a feature and splits the data based on a randomly chosen split value between the minimum and maximum values of the feature. The process is repeated recursively to create a binary tree. Since anomalous points are rare and tend to differ significantly from the rest of the data, they tend to be isolated quicker (i.e., closer to the root of the tree). Points that require more splits to be isolated are considered normal, while those isolated with fewer splits are flagged as anomalies.

OCSVM [12] is another popular method used in unsupervised anomaly detection. It is an extension of the traditional SVM but adapted for situations where only one class (normal data) is available during training. The model learns a decision boundary around the normal data, and points that fall outside this boundary are classified as anomalies. OCSVM works well in cases where the anomalies are relatively far from the normal class, though it may struggle when anomalies are subtle or clustered with normal points.

Clustering techniques are another common approach in unsupervised learning. K-Means is a simple and widely used clustering algorithm that partitions the dataset into k clusters. Each data point is assigned to the nearest cluster center, and anomalies are detected as points that are far from any cluster center. However, K-Means may not perform well when clusters are not spherical or when the data has varying densities. DBSCAN is another powerful clustering method that detects anomalies by identifying low-density regions in the data [13]. DBSCAN groups data points into clusters based on their density, and points that do not belong to any cluster are flagged as anomalies. Unlike K-Means, DBSCAN does not require specifying the number of clusters and can detect arbitrarily shaped clusters, making it more robust in complex datasets.

Long Short-Term Memory (LSTM) Networks are another powerful approach to unsupervised anomaly detection, particularly for time series data. LSTMs,

a type of Recurrent Neural Network (RNN), are designed to capture long-term dependencies in sequential data. In anomaly detection tasks, LSTMs are often trained as sequence-to-sequence models or autoencoders to learn the normal patterns in time series. When the model is deployed, significant deviations between the reconstructed or predicted sequences and the actual data are flagged as anomalies. For instance, LSTMs have been successfully applied to detect anomalies in electrocardiogram (ECG) signals, network traffic data, and industrial sensor data, demonstrating their ability to handle complex, non-linear patterns [14]. Their flexibility and robustness make LSTMs particularly effective for applications where the nature of anomalies is not well-defined.

In addition to unsupervised techniques, there is also interest in semi-supervised learning, which falls between supervised and unsupervised learning. Semi-supervised methods assume the availability of a small amount of labeled data, often consisting of normal instances, along with a large amount of unlabeled data. These methods can leverage the labeled data to improve the detection of anomalies. For example, a Semi-Supervised SVM can train on labeled normal data while simultaneously incorporating the structure of the unlabeled data to detect outliers.

In summary, unsupervised learning techniques are invaluable in real-world anomaly detection where labeled data is limited. By leveraging the inherent structure and distribution of the data, these methods can detect patterns that deviate from normal behavior, even without the need for labeled anomalies.

Deep Learning

Deep learning can be used for anomaly detection, as it allows models to automatically learn complex patterns in data ([2] provides a review on the use of deep learning in this field). Moreover, it has been applied in the medical domain, where it has been used to identify anomalies in physiological signals, medical images, and patient records, proving to be a valuable tool for early diagnosis and monitoring of health conditions ([15] reviews its application in this context).

Autoencoders and Generative Adversarial Network (GAN) have emerged as powerful tools for detecting anomalies in complex datasets, especially in high-dimensional spaces [16]. Autoencoders, for instance, are neural networks designed to learn a compressed representation of input data by encoding it into a lower-dimensional space, then reconstructing it back to the original. The key idea is that normal data points will have low reconstruction errors, as the autoencoder has learned to represent them well, while anomalies will have higher reconstruction errors, since the model struggles to accurately recreate these rare or unusual instances. Anomalies are thus detected by thresholding the reconstruction error [3].

Another deep learning-based technique for anomaly detection is the use of GANs.

In a GAN, two neural networks—the generator and the discriminator—are trained simultaneously in a game-like setting, where the generator attempts to create synthetic data points that resemble the normal data, while the discriminator tries to distinguish between real and generated data. For anomaly detection, the trained discriminator can be used to detect anomalies, as it is adept at identifying data points that deviate from the learned normal distribution [17]. The generator can also be used to generate potential anomalies, further aiding the detection process. GANs are particularly useful when labeled data is scarce, as they leverage unsupervised learning techniques to model normal behavior.

Additionally, Recurrent Neural Network (RNN)s, particularly Long Short-Term Memory (LSTM) networks [18], are frequently used for anomaly detection in time-series data. LSTMs are well-suited to capture temporal dependencies in sequential data and can predict future values based on past observations. Anomalies are detected when the predicted values deviate significantly from the actual observations. This method has proven effective in various domains, such as medical signal processing and network security, where time dependencies are crucial [3].

Deep learning models like autoencoders, GANs, and LSTMs have been widely adopted due to their ability to learn intricate patterns in complex, high-dimensional datasets. However, one of the key challenges with these models is their interpretability. While they offer powerful tools for anomaly detection, especially in scenarios where traditional statistical models fall short, understanding why a particular data point is labeled as an anomaly can be difficult due to the "black-box" nature of deep learning models [16]. As a result, there has been increasing interest in developing more interpretable models and integrating explainability techniques in deep anomaly detection.

2.1.3 Time Series Anomaly Detection

Detecting anomalies in time series data is particularly challenging due to the temporal dependencies. Techniques such as ARIMA, LSTM networks, Transformer models, and even methods like OCSVM and Isolation Forest are frequently employed to capture temporal patterns and identify outliers that deviate from expected trends [17]. While ARIMA and LSTM excel at modeling temporal dependencies, OCSVM and Isolation Forest are well-suited for detecting anomalies in more complex, high-dimensional time series data.

Sliding Window Techniques

In time series anomaly detection, preprocessing methods such as sliding window techniques play a crucial role in simplifying the time series data, enabling more efficient anomaly detection. The sliding window approach segments the time series

into overlapping or non-overlapping windows of fixed size, allowing local patterns to be captured and anomalies to be detected within each segment. This technique is particularly valuable for real-time monitoring, as it facilitates incremental processing of time series data without needing to analyze the entire dataset at once [19].

For instance, sliding windows can be combined with machine learning algorithms to predict the next values in a series, and anomalies are flagged when the actual values deviate significantly from the predicted ones [20, 21]. In wireless sensor networks, an adaptive sliding window technique has been proposed, where the window size is dynamically adjusted based on the trend of the time series, improving the detection accuracy of pattern anomalies [22]. Similarly, in hydrological time series, sliding windows have been employed to detect outliers by segmenting the data and using autoregressive models to predict future values within each window [19, 20].

Sliding window techniques are also effective when combined with data compression methods. For example, the Window Delta (WD) algorithm applies sliding windows to reduce data volatility and enhance the compression ratio in time series compression, thus improving the efficiency of both storage and transmission in large-scale time series data [23]. Moreover, sliding windows can be used to facilitate motif discovery, where recurring patterns in time series are identified by segmenting the data and comparing subsequences [24].

2.1.4 Anomaly Detection with Healthcare Data

Anomaly detection in healthcare data has become increasingly critical as it can help identify abnormal patterns that may indicate potential health risks, equipment malfunctions, or errors in medical records. Common applications include detecting irregularities in medical signals such as ECG and EEG, where anomalies can be indicative of heart conditions or neurological disorders.

Machine learning models like OCSVM, Isolation Forest, and deep learning methods such as Autoencoders and LSTM networks are frequently used for anomaly detection in healthcare. These models are capable of learning from large amounts of historical patient data to detect subtle deviations in new patient data, often flagging rare events or outliers that may require further investigation. For instance, LSTM networks are particularly useful in processing time series data, such as monitoring heart rates or glucose levels over time, and identifying deviations from expected trends, without letting anomalies set off alarms [3, 17].

In addition, deep learning models like GANs have recently been explored for generating synthetic healthcare data to improve anomaly detection in cases where real anomalies are scarce. These models enable the generation of diverse examples of normal data, against which rare anomalies can be better detected [16].

The ability to detect anomalies in healthcare data is crucial for early diagno-

sis and preventive care. With the increasing integration of wearable devices, even though these are not the most common tools employed in healthcare or Internet of Things (IoT) technologies, among others, healthcare providers are now able to continuously monitor patients and receive alerts when anomalies are detected, improving patient outcomes through real-time intervention [22].

2.2 Change Point Detection

Change point detection is an essential method in time series analysis, allowing the identification of points where the statistical properties of the data change abruptly. These changes, often referred to as regime shifts, can result from external events or internal transitions in the data-generating process. Detecting change points is critical in numerous domains, such as medical diagnostics, finance, climate science, and manufacturing, where early detection of significant shifts can lead to timely interventions and better decision-making.

Change point detection methods can be broadly classified into two categories: **offline** and **online** approaches [25]. Offline methods analyze the entire dataset retrospectively, making them ideal for studying historical data where the objective is to understand past transitions. These methods are commonly used in applications such as financial market analysis and environmental studies. On the other hand, **online methods** detect changes in real time as new data becomes available. This real-time capability makes them particularly suited for scenarios requiring immediate responses, such as patient monitoring in ICU or automated fault detection in industrial systems.

Change point detection methods can also be categorized by their underlying methodology. **Statistical methods**, such as Cumulative Sum (CUSUM), likelihood ratio tests, and kernel-based approaches, rely on probabilistic principles and assumptions about the data's distribution. They are computationally efficient and interpretable but may struggle with high-dimensional or complex datasets. In contrast, **machine learning methods**, including clustering algorithms and deep learning models like LSTMs, are data-driven and more flexible, making them effective for handling non-linear, noisy, and high-dimensional data. While statistical methods provide a solid foundation, machine learning approaches are increasingly favored for tackling more complex time series problems.

2.2.1 Statistical Methods

Statistical methods for change point detection are widely used due to their simplicity and interpretability, making them suitable for various applications such as climate science, finance, and medical diagnostics. These methods can be categorized into

parametric and **non-parametric** approaches.

Parametric Methods

Parametric methods rely on predefined assumptions about the data's distribution, such as normality. These methods focus on detecting changes in key statistical properties like mean or variance, making them efficient and interpretable for structured datasets. Two prominent techniques in this category are:

CUSUM (Cumulative Sum) CUSUM is a sequential method designed to detect changes in the mean of a time series. It calculates the cumulative sum of deviations from an expected mean:

$$S_t = \sum_{i=1}^{t} (x_i - \mu),$$

where x_i is the data point at time i, and μ is the expected mean. When the cumulative sum S_t exceeds a predefined threshold, it signals a potential change point. This method is computationally efficient and widely applied in signal processing, particularly for monitoring physiological signals such as heart rate and blood pressure in ICU settings. For example, abrupt shifts in heart rate patterns can be detected early, enabling timely medical interventions [26].

Likelihood Ratio Test The likelihood ratio test compares the likelihood of the time series under two models: one assuming no change and the other assuming a change point exists. The test statistic is given by:

$$\Lambda = 2\left(\log L_1 - \log L_0\right),$$

where L_1 is the likelihood under the change-point model, and L_0 is the likelihood under the no-change model. This method is effective in detecting shifts in the mean or variance of a time series and is particularly useful in financial applications, such as identifying sudden market volatility or price changes. Its reliance on distributional assumptions, however, limits its flexibility in noisy or complex datasets [27].

Parametric methods are efficient and interpretable but may fail to capture changes in data with non-Gaussian or unknown distributions, which is where non-parametric methods excel.

Non-Parametric Methods

Non-parametric methods avoid making assumptions about the underlying distribution of the data, making them more flexible for a wide variety of time series. These methods are particularly suitable for high-dimensional, non-linear, or irregular datasets. Key approaches include:

Kernel-Based Change Point Detection Kernel-based methods use a kernel function, such as the Gaussian kernel, to map the data into a higher-dimensional space. Changes are detected by measuring the similarity between segments in this feature space. A common formulation involves maximizing a cost function based on kernel embeddings to locate the change points. This method is particularly effective for detecting subtle changes in biomedical signals, such as EEG or ECG data, where the distributional properties are unknown. For example, kernel-based methods can identify small but meaningful shifts in neural activity patterns, which are critical in diagnosing neurological conditions [28]. However, the computational intensity of this approach can be a limitation when dealing with large datasets.

Bottom-Up Change Point Detection (CPD) The Bottom-Up method, implemented in the Ruptures library, is a hierarchical segmentation algorithm [25]. It begins with the maximum number of segments (one per data point) and iteratively merges adjacent segments to minimize a cost function, such as squared error:

$$\text{Cost} = \sum_{i=1}^{k} \text{Error}(S_i),$$

where S_i represents each segment. This method is particularly advantageous when the number of change points is unknown, as it dynamically determines the optimal segmentation. Bottom-Up CPD has been applied to long-duration signals, such as temperature or motion data, to identify meaningful shifts in trends or variance. For instance, in ICU settings, this method can detect gradual changes in a patient's physiological state, such as increasing instability in respiratory patterns, aiding in early diagnosis and treatment [25].

Non-parametric methods are highly adaptable and robust but require careful consideration of computational costs, especially when applied to real-time systems or large datasets.

Both parametric and non-parametric statistical methods are invaluable for detecting change points in signals, with each offering unique advantages. Parametric methods like CUSUM and likelihood ratio tests are computationally efficient and well-suited for structured data with known distributions, while non-parametric methods like kernel-based detection and Bottom-Up CPD provide the flexibility needed for complex, high-dimensional data. These methods have shown significant utility in monitoring physiological signals, detecting anomalies in medical diagnostics, and analyzing financial or environmental data. Despite their strengths, the challenges of scalability and sensitivity to assumptions highlight the potential benefits of combining statistical techniques with machine learning approaches.

2.2.2 Machine Learning Methods

Machine learning methods for change point detection leverage data-driven models to identify transitions in time series, often without relying on strict assumptions about the data's underlying distribution. These methods are particularly effective in handling non-linear, high-dimensional, and noisy datasets, making them suitable for complex real-world applications such as healthcare, finance, and industrial monitoring.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) Networks LSTMs, a specialized type of RNN, are widely used for sequential data due to their ability to capture long-term dependencies in time series. In the context of change point detection, LSTMs are trained to predict the next value in a sequence, and deviations between the predicted and actual values can indicate potential change points. This method is particularly effective in detecting subtle or non-linear changes in complex datasets, such as electrocardiogram (ECG) signals or stock prices [14]. Additionally, LSTMs can be integrated into hybrid models, combining statistical techniques with deep learning for improved accuracy.

Clustering-Based Methods Clustering algorithms, such as K-means or Gaussian Mixture Models (GMMs), are used to segment time series data into homogeneous regions. Change points are identified at the boundaries between clusters, where the statistical properties of the data differ significantly. These methods are computationally efficient and can handle diverse types of time series but may struggle with highly dynamic datasets unless paired with advanced feature extraction techniques [29].

Transformers for Change Point Detection Transformers, originally developed for natural language processing, have recently been adapted for time series analysis due to their ability to model long-range dependencies efficiently. By using attention mechanisms, transformers can focus on the most relevant parts of a sequence, making them highly effective for detecting changes in large-scale, complex datasets. For example, they can be applied to multivariate time series in medical monitoring to identify shifts in patient conditions [30].

Hybrid Models Hybrid approaches combine the strengths of statistical methods with machine learning techniques. For instance, statistical methods like kernel-based change point detection can be used to pre-process the data, while machine learning models like LSTMs or transformers handle the final detection. These hybrid methods have shown significant improvements in both accuracy and interpretability, particularly for applications involving noisy or irregular data.

Machine learning-based methods represent a powerful complement to traditional statistical techniques, offering greater flexibility and robustness in complex scenarios. However, their increased computational cost and need for large labeled datasets can pose challenges, particularly in real-time applications.

Chapter 3

Data analysis

In this chapter, we will focus on showing how we have addressed the various challenges we faced with regard to the data with which we worked with. We will explain the characteristics of the dataset we are working with, the problems encountered, and the analysis of the different signals within the dataset.

3.1 Problems with availability of the data

There was an issue with obtaining the rights to access the hospital's data which unfortunately was not resolved in time for us to be able to use them during our project. Therefore, in this project we had to work with an external open source database, which resembled the data from the hospital, so that our work may be integrated in their system some day.

The data set with which we work is VitalDB, data was obtained from surgical patients who underwent routine or emergency surgery in 10 of 31 operating rooms of Seoul National University Hospital (Seoul, Republic of Korea) [31, 32].

This data set is similar to the data set that the Hospital Niño Jesús was going to provide us with, but with some slight differences such as the fact that the data is extracted from sensors in an operating room instead of reat time data coming from the ICU, which was the original idea for this project.

3.2 Description of the dataset VitalDB

As noted above, we have worked with an open-access database called VitalDB which is a real-world database that contains physiological signs and vital signs from patients. As explained in [32], the dataset consists of intraoperative vital signs data and perioperative clinical information of 6,388 cases. Vital signs data includes up

to 12 waveform and 184 numeric data tracks acquired from multiple anesthesia devices applied to patients during surgery. The total number of data tracks is 486,451 (average 87, range 16–129). Data is not pre-processed because the real-world noise in the vital signs data is very essential to the development of practical monitoring algorithms. Therefore, this dataset is a valuable alternative to develop this project due to the missing data set from Hospital Niño Jesús because of legal restrictions related to data privacy of the patients. The dataset contains signals such as

- **ECG (Electrocardiogram)**: captures the electrical activity of the heart.
- **Blood Pressure**: measured from arterial lines or non-invasive monitoring devices.
- **Oxygen Saturation**: monitors the percentage of oxygen in the blood

The data is continuously collected over time, providing time-series information on the patient's status during their stay in the hospital. Often in care units or during surgeries.

It is a really large database, so for the initial tests and attempts, we worked only with the first record (0001.vital), which is a fraction of the whole VitalDB data set. However, in our final attempts, we will use several records to obtain reliable results, as working with a single record would not be good practice.

The ECG signal captures the electrical impulses of the heart and is a direct measure of the heart's function, while the Heart Rate is derived from this signal.

The data itself had approximately 5.7 million rows, each one representing a different time point in the patient's monitoring. The high volume of the data allows for detailed analysis but also poses challenges since it has a lot of missing values.

3.3 Challenges with the Dataset

Although the dataset provided valuable insights for preliminary analysis and testing, we encountered several challenges, primarily due to substantial amounts of missing data. Both the ECG and Heart Rate variables had significant gaps; for instance, the Heart Rate column exhibited 99 percent missing values. However, this still left us with approximately 5,000 usable data points after handling the NaNs, which was sufficient to conduct exploratory analysis and test different techniques.

It is important to note that these gaps are not due to errors within the dataset but are a result of clinical practices. In many cases, medical staff may not have all sensors activated continuously, as they may selectively activate them based on clinical necessity. Consequently, this leads to periods without recorded signals, con-

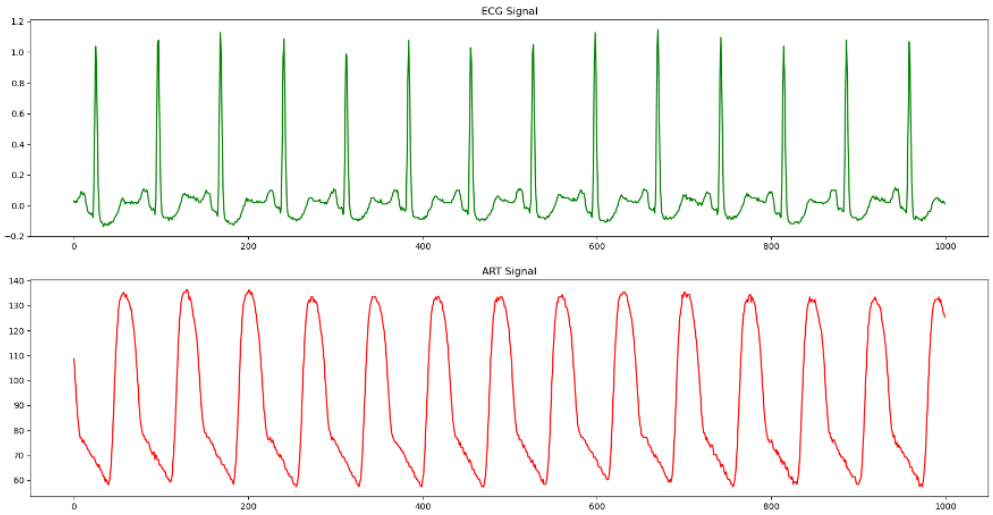

Figure 3.1: The signals, an example given by the library itself

tributing to the observed data gaps, since they belong to the same time session for a patient.

The ECG signal was affected by noise, which is common but this noise needed to be accounted for also when performing the anomaly detection, most of the times by normalizing the data.

3.4 Time representation and Frequency

VitalDB is inherently time-series based, with each measurement associated with a specific timestamp. However, as is a hospital's data, this means that the frequency is not always consistent. For instance, the ECG signal is capture continuously with very high frequency while others (like heart rate) can have significant gaps.

This added a layer of complexity to the analysis, even more when trying to align or compare different metrics. Since this was a preliminary test of techniques, we focused primarily on the number of signals rather than their precise timestamps.

3.5 Focusing on other signals, based on the Hospital's feedback

After the midterm report, during one of our meetings with the hospital, we were provided with feedback on the most relevant physiological signals that we focused on during the second part of the project.

- **Heart Rate Signals**: Includes **FC_PULSO** (Solar8000/HR), **FC_ECG**, and

FC_TA_INV (Solar8000/PLETH_HR) for comprehensive heart rate data.

- **SAT_O2** (Solar8000/PLETH_SPO2): Measures oxygen saturation, important for assessing respiratory efficiency.
- **FI_O2** (Solar8000/FIO2, Primus/FIO2): Fraction of inspired oxygen, crucial for monitoring patients under respiratory support.
- **Respiratory Rate Signals**: Includes **FR_RESPIRAT_ECG** (Solar8000/RR), **FR_RESPIRAT_CAP** (Solar8000/RR_CO2), and **FR_RESPIRAT_VENT** (Solar8000/VENT_RR) for monitoring breathing patterns.
- **CO2 Levels**: Includes **INS_CO2** (Solar8000/INCO2) and **ET_CO2** (Solar8000/ETCO2, Primus/ETCO2) for carbon dioxide monitoring.
- **Ventilator Parameters**: Includes **PIP** (Solar8000/VENT_PIP, Primus/PIP_MBAR), **PEEP_P** (Solar8000/VENT_MEAS_PEEP), and **COMPLIANZA** (Solar8000/VENT_COMP for assessing ventilator settings and lung compliance.

The hospital's staff recommended for us to focus especially on those signals measured by the Solar8000 and Primus devices, as those were the most similar to the signals contained by the data set of the hospital and the ones that we could extract the most promising insights from.

Given that we never obtained the actual data from the hospital, we worked with the VitalDB data set throughout the whole project, focusing on signals like heart rate during the second half of the project.

Chapter 4

Data preprocessing

After analysing the VitalDB dataset, the first step involved choosing the features or signals we were going to work with.

In our first approaches, as we did not know which signals our actual data from Hospital Niño Jesús would have, we chose to work with different signals for each implementation, but we opted to use the signals that did not have or had very few missing values, such as SNUADC/ART or SNUADC/ECG_II , where, SNUADC stands for Seoul National University Analog-to-Digital Converter [32], which refers to a system developed by Seoul National University to capture physiological signals through an analog-to-digital conversion process, allowing for continuous monitoring of patient data, such as arterial blood pressure or the second lead of a standard electrocardiogram.

But as we progressed through the project, we learned that the missing values was due to the nature of a clinical context, where sensors are not connected the whole time to the patient, and in addition to this point, after the recommendations from the hospital staff related to the most important signals to analyze in the data set, we decided to focus on signals measured by Primus and Solar8000 (such as heart rate). For this reason we changed the selection of employed signals during the course of our project, given that initially we did not have this feedback.

Other steps that we have always followed for all our processes and we will keep following are the data subsampling and normalization.

Due to the high volume of data, we considered that a very healthy practice is to take only a small percentage of it, such as 0.1% or 1%, to reduce the dataset to a manageable size without compromising the integrity of the signals. By doing this we make sure that the model training will be faster, and this will help us a lot in the first attempts, where we are just trying to figure out which one is the best way to solve our anomaly detection problem. However, in the future we will use more percentage of the data, or even the complete data, for our final model, depending on our consideration.

On the other hand, normalizing the data to ensure all features operated on comparable scales, is crucial for the performance of machine learning models. Specifically, we utilized Min-Max scaling to normalize the ECG values in one of our LSTM models, but we used other techniques for some of the other models that we tried, as it will be explained later.

After some initial attempts, we decided that the best way to format our data as input (X) for training our models was to reshape it into a matrix of sequences, where each sequence contains n samples. Initially, we considered segmenting the data based on time intervals. However, since the signals we are working with have a constant time between samples, we opted to segment the data by the number of samples instead.

To handle the last few data points that do not fit perfectly into a sequence of the same length as the others (with the same overlap), we decided to simply leave them out and ignore them. This way, we avoid forcing the addition of an extra sequence with more overlap than the rest, which could lead the model to assign more weight to those data points compared to the rest of the sequences.

This approach led us to another question: what is the optimal sequence length and overlap? Since we didn't know the best values upfront, we decided to proceed by simultaneously experimenting with data reformatted with different sequence lengths and overlaps. We would then select the best performing configuration.

From these tests, we drew several conclusions. For example, we found that excessively long sequence lengths resulted in the detection of very few anomalies, while very short sequence lengths caused the detection of too many anomalies. Therefore, we plan to aim for a middle ground in future iterations to balance detection sensitivity.

After our first approaches, we focused more on the signals that the hospital dataset includes, to make it possible for them to implement the models and methods that we create. The problem with this is that many of them include a lot of missing values, such as the Solar8000/HR signal. To deal with it, we first identified and removed all rows with missing values using the `dropna()` function. Afterward, we analyzed the time differences between consecutive samples to ensure a consistent interval of 2 seconds, as irregular time intervals could impact the model's performance. We located the first index where a stable and consistent time difference was maintained and extracted a continuous segment of the signal starting from that point. Finally, we selected a specific portion of this clean segment with a fixed length of 5451 samples, ensuring that the data used for model training and evaluation was both complete and consistent. We used processes similar to this one with other signals that presented many missing values.

Chapter 5

Anomaly detection

As we have mentioned, anomaly detection plays a crucial role in identifying unexpected patterns or deviations in data that do not conform to expected behavior. In the context of this project, anomaly detection is applied to time series data collected from ICU sensors to distinguish between normal fluctuations and potentially critical abnormalities in patient vitals. This chapter outlines the methods that we explored for anomaly detection, detailing the techniques implemented, their underlying principles, and the specific steps taken to adapt them to the clinical setting and our data.

5.1 One-Class SVM model implementation

One of the models applied in our anomaly detection pipeline was the OCSVM. This model is widely recognized in state-of-the-art anomaly detection tasks for its effectiveness in data sets where the majority of data points represent normal behavior, and only a small fraction are outliers. The OCSVM works by learning a boundary in the feature space that separates normal data from potential anomalies. For this test, we applied the model to the clinical signals Solar8000/HR and Solar8000/ART-SBP, which are critical for cardiovascular monitoring and detecting abrupt deviations that could indicate physiological abnormalities.

The input data was pre-processed using standard scaling, ensuring that all features were normalized to have a mean of zero and unit variance. This step is crucial especially for SVM-based models, as they are sensitive to the scale of the input features. We trained the OCSVM with a radial basis function (RBF) kernel, setting the hyperparameter nu to control the proportion of outliers the model would allow. The output of the model was a binary prediction for each data point, where 1 represented normal behavior and -1 indicated an anomaly. Furthermore, we calculated the L2 deviation of the detected anomalies from the learned boundary to filter out minor deviations and focus on significant anomalies using a defined threshold, as it

was important to take into account that there cannot be several anomalies in the context of a hospital, as potential threads tend to be quite uncommon.

To visualize the performance of the model, we generated graphs for each signal, highlighting significant anomalies in red over the original signal. These visualizations provide an intuitive understanding of the model detections.

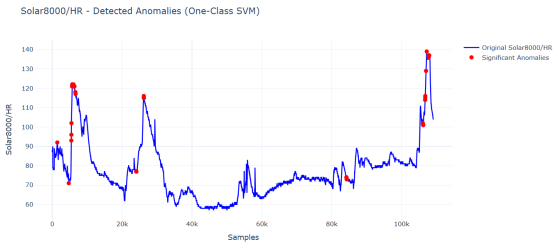

Figure 5.1: Detected anomalies in HR

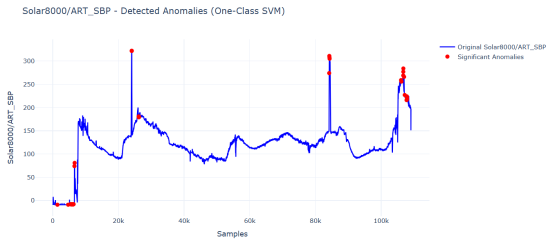

Figure 5.2: Detected anomalies in ART_SBP

The results from the experiment showed that the model successfully identified several anomalies in both signals. Initially, the model identified a certain number of anomalies; as we applied the deviation threshold, we were able to retain only the most significant anomalies. For the Solar8000/HR signal, the OCSVM detected rapid changes and spikes in heart rate, particularly in regions with sharp transitions. These anomalies likely correspond to events such as sudden tachycardia or irregular heart rate patterns, which could indicate potential cardiovascular events. In the case of the Solar8000/ART-SBP signal, anomalies were identified during periods of extreme systolic pressure values, such as sharp increases or abrupt drops. These anomalies likely highlight critical clinical events, such as hypertensive episodes or fluctuations in arterial compliance.

The generated graphs also include a zoom feature, allowing for a closer inspection of the detected anomalies. In this way it is easier to analyze a certain anomaly and verify it is truly an anomaly or a false positive detection.

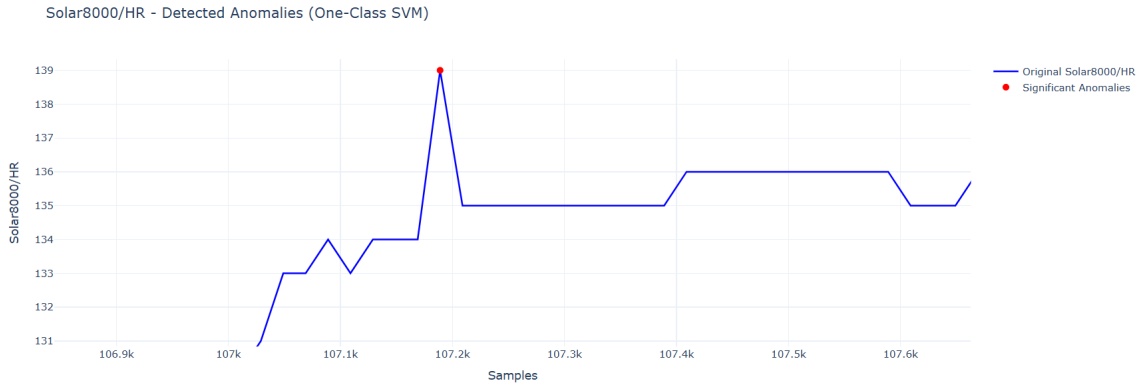

Figure 5.3: Zoomed anomaly with One-Class SVM

However, we noticed that these models primarily distinguished between data that followed the normal trend and data that deviated significantly from it. As a result, models like OCSVM exhibited significant limitations in effectively detecting anomalies. Specifically, periodic anomalies present in the data might be misclassified as normal, reducing the ability of the model to accurately identify such patterns. Although the OCSVM model has provided us with useful results, we believe this approach is not the most suitable for our objectives, as our dataset is unlabeled.

5.2 Isolation forest model implementation

Initially, before implementing the sliding window technique to sequence the data, we experimented with several models, including the Isolation Forest. However, the results did not meet our expectations. Without properly sequencing the data, the model struggled to capture the temporal patterns inherent in time-series data. As a result, it failed to detect anomalies in a reliable manner.

After deciding to sequence the data through a sliding window, as suggested during our review, we revisited the Isolation Forest model. Using the sliding window approach, we segmented the signals into sequences of varying lengths and overlaps to better capture the temporal structure of the data. We implemented the model on the SNUADC/ART signal, which contains no missing values, making it an ideal candidate for our initial tests. The sliding window allowed us to create a matrix of sequences, where each sequence represented a fixed number of samples from the original signal.

For our tests, we used various sequence lengths (50, 100, 200, 500, 1000, 2000, and 5000 samples) combined with different overlap values (10, 20, 50, 100, 200, 500 samples), resulting in 32 different datasets. Each dataset was normalized using a Standard Scaler to ensure consistency across different sequences. We then

applied the Isolation Forest model with three different contamination levels (0.01, 0.02, and 0.05) to each dataset, producing a total of 96 graphs representing the detected anomalies. This approach allowed us to explore how different combinations of sequence length and overlap impacted the model's sensitivity to anomalies.

The following figure shows an example of the original signal and the anomalies detected by the Isolation Forest in the case where we used a sequence length of 100 samples and an overlap of 20 samples:

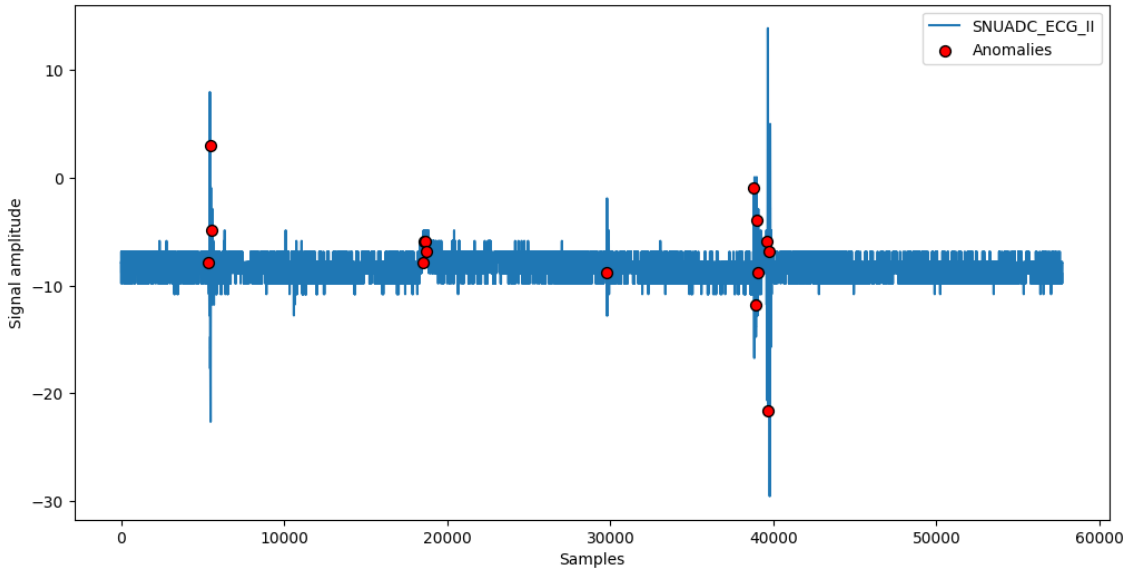

Figure 5.4: Detected anomalies in SNUADC/ART signal using Isolation Forest (Sequence length: 100, Overlap: 20)

To better analyze the anomalies, we zoomed into specific sections of the signal where anomalies were detected. The next figure provides a closer look at two identified anomalies—one that seems to clearly indicate abnormal behavior and another that is less clear, potentially a false positive. This detailed view helps us evaluate the accuracy of the model's predictions and will be useful when presenting the results to medical professionals for further validation.

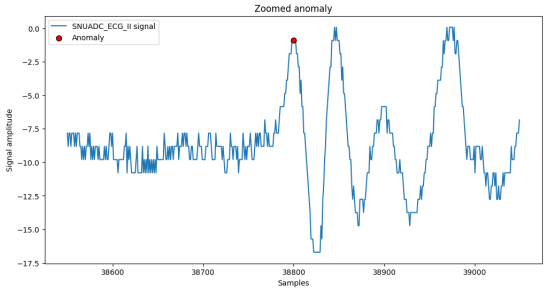

Figure 5.5: Example of a "clear" anomaly

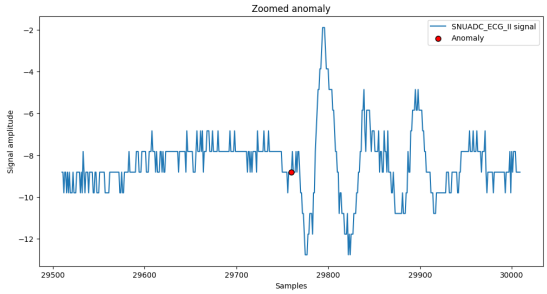

Figure 5.6: Example of a rare anomaly

In these process, we encountered some challenges. For example, datasets with very large sequence lengths (such as 1000 or 5000 samples) tended to result in very few anomalies being detected, while smaller sequence lengths (such as 50 samples) produced a much larger number of anomalies. This variation made it difficult to identify an optimal configuration without the input of domain experts, such as medical staff, to determine which anomalies are clinically relevant and which are false positives.

After the hospital communicated us the signals they had in their dataset, we decided to apply this method to a signal present in their data, and we chose Solar8000/HR. So, we applied isolation forest with all the conclusion we had already extracted, and we did it more straightforward: we chose a sequence length of 100, with 20 overlap and 0.05 contamination, and got the these results:

Although the Isolation Forest model provided us with interesting insights, we have decided to set it aside for now. Similar to the OCSVM, this method may not be the most suitable for our current objectives, as it relies on unsupervised learning without specific knowledge of what constitutes a true anomaly in our dataset. Given the limitations we encountered, we will focus on refining other models, such as LSTM or Seasonal AutoRegressive Integrated Moving Average (SARIMA), that may be better suited to the complexities of our dataset.

5.3 LSTM model implementation

As discussed in the State of the Art section, LSTM networks are highly effective for anomaly detection in time-series data due to their ability to capture temporal dependencies. LSTM models are particularly suited for predicting future values based on past observations, and anomalies are identified when the predicted values deviate significantly from the actual ones. So, we decided to implement an LSTM model to detect anomalies in our dataset.

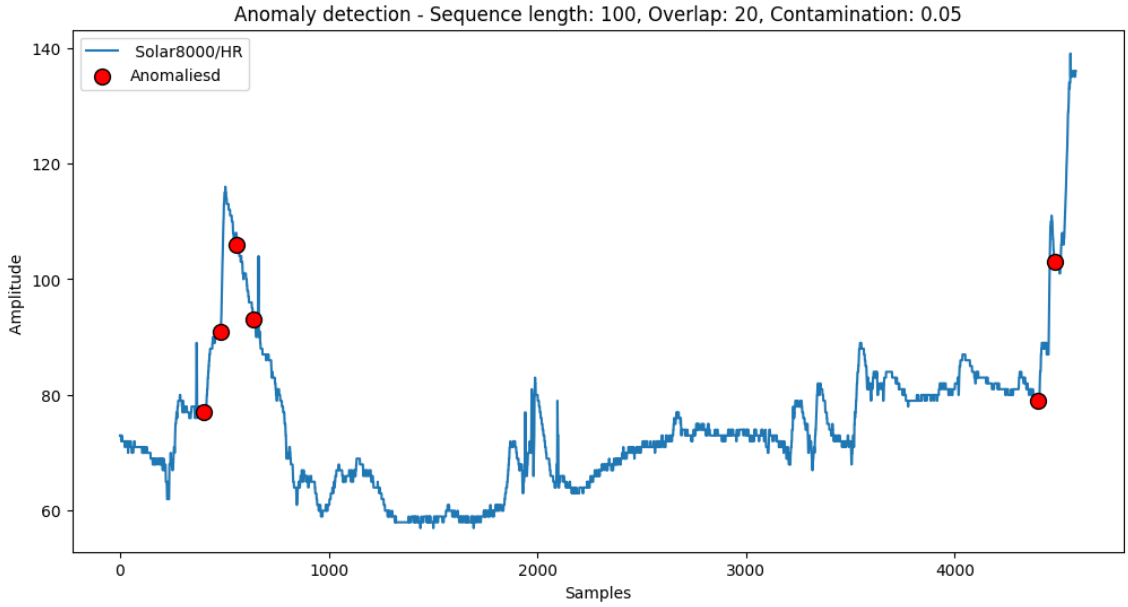

Figure 5.7: Detected anomalies in Solar8000/HR signal using Isolation Forest (Sequence length: 100, Overlap: 20)

Our initial approach involved using a basic LSTM model with two layers, but after evaluating the results, we decided to expand the model architecture to improve its ability to capture complex temporal relationships in the data. The final version of our model consists of five LSTM layers, each with 50 units, and dropout layers were added to prevent overfitting. Specifically, we applied a 0.2 dropout rate after each LSTM layer, meaning 20% of the nodes were randomly ignored during training to help the model generalize better.

To prepare the data for training, we normalized the ECG values using Min-Max scaling to ensure that all features operated on a comparable scale, which is critical for the performance of deep learning models. The LSTM model was compiled using the Adam optimizer and the Mean Squared Error (MSE) as the loss function. We trained the model for five epochs with a batch size of 64, which allowed for efficient learning while keeping computational demands manageable.

To evaluate the performance of the LSTM model, we plotted its predictions against the actual values from the SNUADC/ECG_II signal. In the graph below, the LSTM predictions (in orange) are compared to the actual values (in blue), allowing us to visualize how well the model captured the normal behavior of the ECG signal. During periods of relative stability, the predictions closely follow the actual values, demonstrating the model's accuracy in these instances. However, in regions where significant deviations occur, the model's predictions diverge from the actual values, indicating potential anomalies.

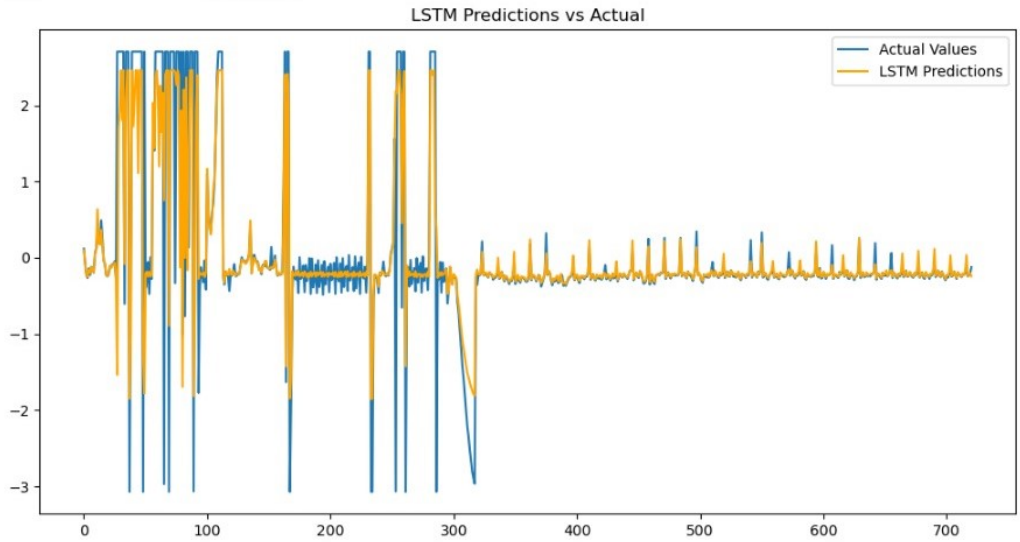

Figure 5.8: Actual value of SNUADC/ECG_II signal vs LSTM predictions

Throughout our experiments, we appreciated that the choice of sequence length and overlap had a significant impact on the sensitivity of the model, the same that happened for all our other models. Longer sequence lengths tended to detect fewer anomalies, whereas shorter sequence lengths resulted in detecting a large number of anomalies, many of which seemed like false positives, in our non-expert eyes.

Based on these findings, we plan to fine-tune the model further, aiming for a balance that allows for more accurate anomaly detection without overwhelming the system with false alarms. This will involve experimenting with moderate sequence lengths to achieve an optimal balance between over- and under-detection.

5.4 Autoencoders

An autoencoder is a type of neural network designed to learn a compressed representation (encoding) of input data while preserving the most relevant information. It consists of two main components:

1. The **encoder**, which compresses the input data into a lower-dimensional latent space retaining the most important characteristics of the data.
2. The **decoder**, which reconstructs the data from this compressed representation, aiming to closely match the original input.

In the context of anomaly detection, an autoencoder is typically trained on normal data, learning to reconstruct it accurately. When presented with anomalies,

which is data that deviates significantly from the normal patterns, the autoencoder struggles to reconstruct it well, resulting in higher reconstruction errors. These errors can then be used to identify potential anomalies. For these tests, we employed two different signals that we considered the most relevant for anomaly detection: Solar8000/HR and Solar8000/ART-SBP, in this way we considered that the different autoencoders could detect the patterns between the signals to reconstruct the original signal to discriminate between normal data and outliers that could potentially be anomalies, as the signals like Solar8000/HR and Solar8000/ART-SBP are likely to have significant relationships between them. However, it could also be interesting to combine not so correlated signals such as SNUADC/ECG-II in order to make the model more robust to generalization and independent data, but it could also bring some drawbacks such as adding noise because it is learning two completely different signals or increase the complexity of the model.

Using multiple related signals, such as Heart Rate (HR) and Arterial Blood Pressure (ART-SBP), enhances anomaly detection by capturing relationships between the signals. These signals often correlate under normal conditions, so deviations in their expected interaction, like an increase in HR without a corresponding rise in ART-SBP, can indicate anomalies that single-signal analysis might miss. By providing additional context, multi-signal models detect inconsistencies more effectively and are more robust to noise, making them especially valuable.

5.4.1 Dense

A dense autoencoder is a neural network that relies on fully connected layers, meaning each neuron in one layer is connected to all neurons in the next. This architecture treats all features equally, without considering spatial or temporal relationships between them.

The model compresses input data into a compressed representation (encoder) and reconstructs it (decoder). During training, it learns to minimize reconstruction error. Dense autoencoders are straightforward to implement and work best for simple patterns, as they lack the ability to capture local or sequential dependencies.

In this implementation, we used a dense autoencoder with three dense layers in the encoder, progressively compressing the input into a latent representation of 16 neurons. The decoder mirrors this structure, reconstructing the input by expanding back to its original dimensions. Batch normalization was applied after certain layers to stabilize training and improve performance.

The model was trained on normal data using mean squared error (MSE) as the loss function employing the MinMax Scaler, with additional techniques like early stopping and learning rate scheduling (ReduceLROnPlateau) to optimize training. Early stopping prevents overfitting by halting training when the validation loss stops

improving for a specified number of epochs. Meanwhile, ReduceLROnPlateau dynamically reduces the learning rate when the validation loss plateaus, helping the model converge in an easier way.

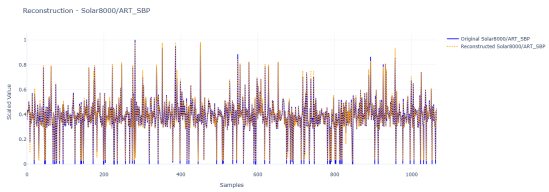

Figure 5.9: Reconstruction (Dense) - Solar8000/ART-SBP

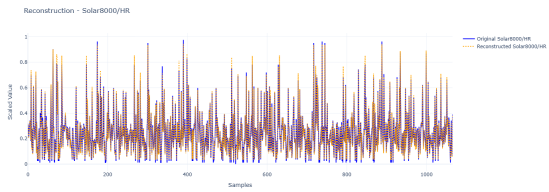

Figure 5.10: Reconstruction (Dense) - Solar8000/HR

As it can be appreciated in the graphs, the signals are very well reconstructed, implying that the autoencoder was able to recreate the signal in an optimal way. Although we could extract some potential anomalies from these graphs, plotting the reconstruction error graph would be easier to see if the model has identified any outliers.

Figure 5.11: Error Reconstruction Dense Autoencoder

The graph shows the reconstruction error from the dense autoencoder, where the blue line represents the error values for each sample and the red dashed line indicates the threshold. Any sample with a reconstruction error above this threshold is flagged as an anomaly.

In this case, the dense autoencoder has identified 11 potential anomalies. These correspond to the points where the blue line exceeds the red threshold, indicating that the model struggled to reconstruct those specific inputs, likely because they deviate significantly from the normal patterns it learned during training. This suggests that these points may represent unusual or unexpected behavior in the signals.

5.4.2 Convolutional Neural Network

A Convolutional Neural Network (CNN) autoencoder is a more advanced model for anomaly detection in clinical signals, leveraging the ability of convolutional layers to detect local patterns. The encoder compresses the signal by detecting repetitive or localized features, such as peaks or transitions, while downsampling the data to reduce its dimensionality. The decoder then reconstructs the original signal by upsampling and reversing the convolutional process. CNN autoencoders are particularly effective for time-series signals like heart rate or respiratory rate, where local anomalies, for example sudden drops, spikes, or irregular oscillations in certain periods, indicate potential issues. By focusing on local relationships in the signal, the model can efficiently detect subtle irregularities that may not be captured by other models or architectures.

The CNN Autoencoder processes time-series signals by learning local patterns and reconstructing them. The encoder uses two Conv1D layers (32 and 16 filters, kernel size 3, ReLU activation) to extract local features, followed by MaxPooling1D layers to downsample and retain critical information. This creates a latent space, a compressed representation of the input that captures essential patterns.

The decoder mirrors the encoder, with Conv1D layers (16 and 32 filters) and UpSampling1D layers to reconstruct the signal. A final Conv1D layer with sigmoid activation generates the output, ensuring normalized values, and a Cropping1D layer adjusts the length to match the original input. The model is trained using the Adam optimizer and minimizes reconstruction error with the MSE loss function, making it effective for detecting local anomalies in time-series data.

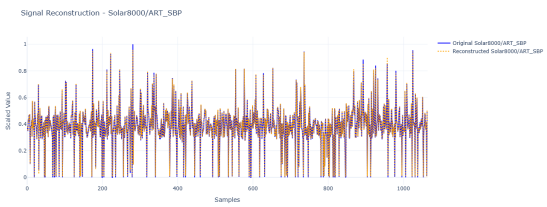

Figure 5.12: Reconstruction (CNN) - Solar8000/ART-SBP

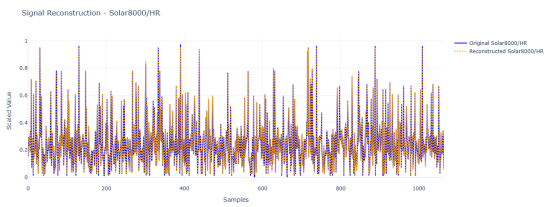

Figure 5.13: Reconstruction (CNN) - Solar8000/HR

Similarly to the dense autoencoder, the model reconstructs the signals, but with the difference that it takes into account local patterns in order to detect drastic changes on data.

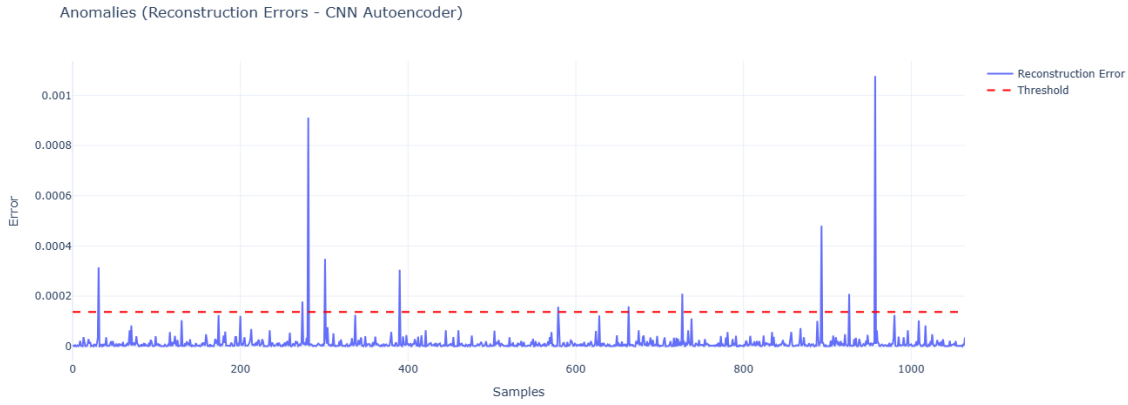

Figure 5.14: Error Reconstruction CNN Autoencoder

As we can see, this model detects fewer anomalies compared to the dense autoencoder, but the identified anomalies are more pronounced. This is likely because the CNN autoencoder is highly sensitive to uncommon local patterns in the data. When the model encounters a pattern that deviates significantly from what it has learned, it classifies it as an anomaly with a sharper distinction, resulting in clearer spikes in the reconstruction error compared to the the rest of the data labeled as normal.

The CNN autoencoder compared to the dense autoenconder identifies in a heavier approach the data that is not clearly common or well reconstructed, for that reason it makes sense that it detects less anomalies but with a higher confidence.

5.4.3 Transformer

A Transformer autoencoder is a modern model designed for several tasks such as NLP, time series modeling or detection of anomalies in several scenarios, especially those with complex patterns or long sequences. Unlike CNNs, which focus on local features, or dense autoencoders, which treat all parts of the signal equally, Transformers use self-attention mechanisms to understand how different parts of a signal relate to each other, both nearby and far apart. The encoder compresses the signal by capturing these all these relationships, while the decoder reconstructs the original signal using this compressed information.

This makes Transformer autoencoders especially useful for clinical signals where anomalies depend on long-term trends or involve relationships between multiple signals.

In our developed architecture, the encoder uses a Multi-Head Attention mechanism to capture both local and long-range dependencies in the input sequence by

learning how different parts of the sequence relate to each other. It also incorporates a Feed-Forward Network (FFN), which applies dense layers to transform and refine the features extracted by the attention mechanism. Layers like Layer Normalization stabilize training, while Dropout prevents overfitting. After the Transformer encoding, a GlobalAveragePooling1D layer is used to compress the time dimension into a fixed-size latent representation, summarizing the most important patterns across the sequence. The decoder reconstructs the sequence using two dense layers followed by a final layer that reshapes the output to match the input dimensions.

Before training, the time-series data is preprocessed into overlapping windows using a sliding window approach to capture temporal patterns effectively. The model is optimized with the Adam optimizer and minimizes MSE to reconstruct the input.

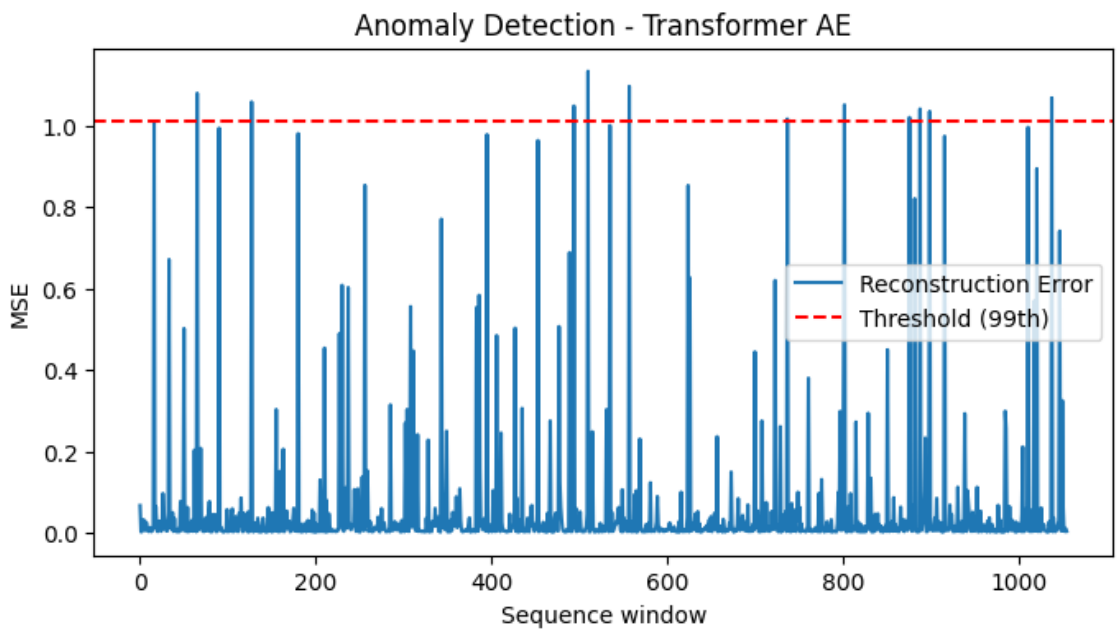

Figure 5.15: Error Reconstruction Transformer Autoencoder

In this result, several outliers are clearly visible above the red line, indicating potential anomalies detected by the model. The Transformer Autoencoder successfully identifies these anomalies by leveraging its ability to capture both local and long-range dependencies in the data, resulting in high reconstruction errors for sequences that deviate significantly from the learned normal patterns. This approach takes advantage of the global context of the signal and the short term dependencies, in that way the model is able to take into consideration a huge range of possibilities in order to detect anomalies.

Comparison between the autoencoder models

After testing different architectures to develop autoencoder models for anomaly detection, we observed distinct advantages and limitations among the approaches.

The Dense autoencoder, while capable of identifying significant anomalies, is a relatively simple architecture. This simplicity results in certain drawbacks, particularly in its ability to detect more subtle or complex anomalies that are not as easily distinguishable.

The CNN autoencoder, on the other hand, excelled by leveraging local patterns to detect potential anomalies. This approach proved particularly effective for clinical signals, where fluctuations typically occur within a defined range. The ability of the CNN to focus on localized features made it an excellent balance between complexity and effectiveness for this context.

Finally, the Transformer autoencoder also delivered impressive results. By combining modern attention mechanisms with simpler components, such as dense layers, this architecture offered a broader perspective of the entire signal while still capturing fine-grained details. This dual capability makes the Transformer autoencoder arguably the most effective approach due to its versatility and potential.

However, the CNN autoencoder remains a strong contender, especially in scenarios involving large datasets where rapid anomaly detection is critical. Its efficiency and ability to meet time constraints make it a highly practical option for such use cases.

Chapter 6

Change Point Detection

For the second goal of this project we needed to explore Change Point Detection (CPD) techniques with a specific focus on vital signs such as heart rate signals. Detecting change points is crucial in identifying significant shifts in physiological states, which could indicate potential health issues or transitions between different patient conditions.

We aimed to apply a range of methodologies, starting with classical statistical models, including parametric and non-parametric approaches, followed by more advanced machine learning techniques. This comprehensive exploration allows us to evaluate the efficacy of various methods in capturing meaningful changes in the data. This section discusses the parametric techniques tested, while subsequent sections cover non-parametric and machine learning methods.

6.1 Statistical Methods

6.1.1 Parametric Models

Parametric models rely on assumptions about the underlying statistical distribution of the data. These models are particularly useful when the data meets these assumptions, offering interpretable and efficient ways to detect change points. In the state-of-the-art review, methods like CUSUM and the Likelihood Ratio Test were highlighted as effective techniques for detecting shifts in statistical properties. Below, we provide a brief introduction to these models and discuss the adjustments that were considered to optimize their performance.

CUSUM

The CUSUM method is a sequential analysis technique designed to detect deviations from a target mean, making it particularly suitable for monitoring gradual shifts in

a signal over time. In this project, the method was applied to smoothed heart rate signals, as smoothing was essential to reduce noise while preserving the signal's statistical properties critical for change point detection.

Key parameters, such as the *window size*, which defines the segment of data considered, and the *threshold*, which determines the sensitivity of detection, were adjusted iteratively during experimentation. Given the absence of ground truth for validation, parameter tuning relied heavily on visual inspection of the results. Observing the plots allowed us to assess the plausibility of detected change points, providing a qualitative measure of the method's effectiveness, which was easier to do for the unexperienced eye than in anomaly detection.

Likelihood Ratio

The Likelihood Ratio Test evaluates the probability of the data under two competing statistical models: the null hypothesis (no change) and the alternative hypothesis (a change has occurred). This method is particularly effective when the statistical distributions before and after a change are well-defined, making it a robust choice for detecting shifts in the heart rate signal.

In this project, smoothing the signal before applying the likelihood ratio test proved critical in reducing noise and enhancing the detection process. Parameters such as the *window size*, defining the range of data considered, and the *threshold*, setting the sensitivity of detection, were iteratively adjusted based on visual inspection of the results, like with CUSUM before.

One of the issues is sometimes several consecutive points were detected as multiple change points, instead of a singel one. To address the challenge of detecting multiple closely occurring change points, clusters of detections were consolidated by averaging their positions. This adjustment reduced redundancy and produced a clearer representation of significant shifts in the signal, as shown in Figure 6.1. While this consolidation improved the interpretability of results, achieving an optimal balance between sensitivity and false positives remains a challenge, emphasizing the importance of fine-tuning the test's parameters.

6.1.2 Non-Parametric Models

Non-parametric methods offer an alternative to parametric approaches by not requiring strict assumptions about the underlying statistical distributions of the data. These methods are particularly appealing when dealing with complex signals where distributional properties are difficult to define. Among the non-parametric approaches, kernel-based CPD methods have demonstrated promising results in this study.

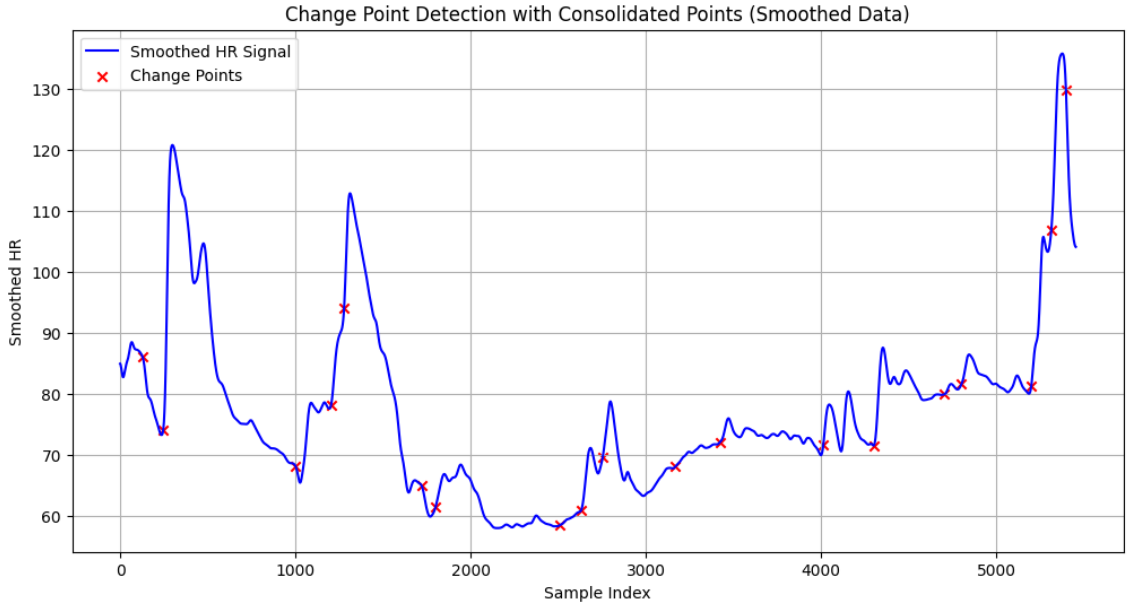

Figure 6.1: Change Point Detection using the Likelihood Ratio Test over Smoothed Data

Kernel-Based CPD

Kernel-based CPD methods rely on calculating a similarity measure between data segments to identify change points. This approach is particularly suitable for detecting changes in complex, non-linear signals, such as heart rate data. In this project, we applied the kernel CPD method to smoothed heart rate signals, as smoothing helps reduce noise while preserving the signal's significant shifts.

The kernel method involves computing a *kernel score*, which quantifies the dissimilarity between adjacent segments of the signal. Parameters such as the *window size*, which defines the segment length used in the calculation, and the *threshold*, which determines the sensitivity of detection, were varied to optimize performance. However, selecting the optimal window size proved challenging, as it significantly influenced the detection results. Without clear medical guidelines, it was difficult to fine-tune these parameters to achieve the best possible outcomes.

To improve interpretability, consecutive change points detected within close proximity were consolidated by averaging their positions. Specifically, points exceeding the threshold and belonging to the same trace were treated as a single change point. This refinement provided a clearer and more interpretable representation of the signal's significant shifts.

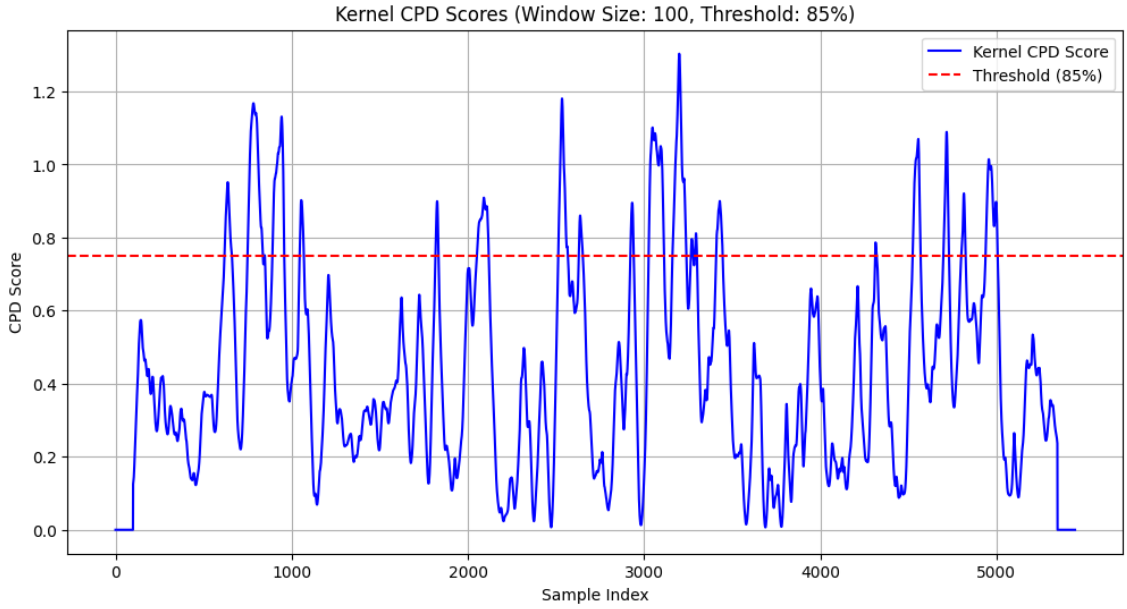

Figure 6.2: Kernel CPD Scores with Window Size = 100 and Threshold = 85%.

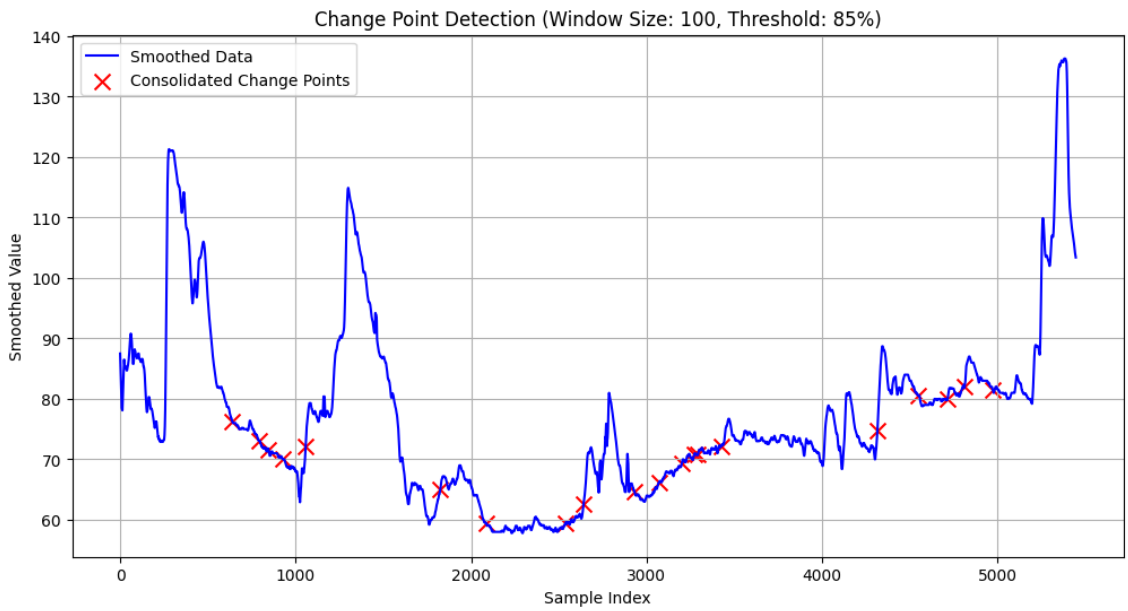

Figure 6.3: Change Point Detection Using Kernel CPD with Consolidated Points (Smoothed Data).

Figures 6.5 and 6.3 illustrate the kernel CPD results. The first figure shows the kernel scores and the threshold used for detection, while the second figure highlights the consolidated change points over the smoothed signal.

Changing parameters like the *window size* substantially affects the shape and outcome of the kernel CPD score. However, without the necessary medical knowl-

edge to validate these adjustments, it remains difficult to ensure that the chosen parameters yield clinically meaningful results.

Therefore, while the kernel CPD method provided reasonable results, further work is required to establish a systematic approach for selecting parameters, particularly the *window size*, as mentioned earlier. Despite these limitations, the kernel CPD method offers a valuable non-parametric perspective for detecting change points in healthcare data.

6.2 Ruptures library

An important tool in this exploration is the ruptures library [25], which is widely recognized for its versatility in CPD. The library offers both parametric and non-parametric methods, making it a valuable resource for analyzing complex signals like those found in healthcare. Its ability to implement classical models such as normal and linear for parametric approaches, alongside non-parametric options like rbf and rank, provides a robust framework for detecting change points across different types of data.

The advantages of using the ruptures library:

- Flexibility: supports multiple models (l1, l2, rbf) and algorithms (PELT, Window, Binary Segmentation, Dynp, K-means...)
- Efficiency: algorithms are optimized for large datasets, ensuring quick computations even with high-resolution signals.
- Built-in functions for visualizing detected change points.
- Customization: parameters like penalty, window-size and segment length can be adjusted to fine-tune the detection process.

In summary, in this section we will develop and analyze different algorithms provided by this library, where we will focus on analyzing the HR signal as recommended by professionals.

6.2.1 Bottom-Up CPD

Bottom-Up CPD is a hierarchical segmentation algorithm from the Ruptures library that begins with the maximum possible number of segments and iteratively merges adjacent segments to reduce their count. This approach is particularly suitable for analyzing signals when the number of changes is unknown, as it provides flexibility in determining the optimal segmentation. In this project, we applied the Bottom-Up

method to detect change points, leveraging its ability to identify changes without requiring prior knowledge of the data distribution.

The Bottom-Up algorithm uses a cost model to evaluate the quality of each segmentation. In our implementation, we used the Radial Basis Function (RBF) model, which calculates a cost based on the dissimilarity between adjacent segments. To determine the optimal number of segments, we computed segmentation costs for different segment counts and plotted a cost curve. We then applied the knee locator method to identify the point where adding more segments yields diminishing returns in cost reduction. If no clear knee is detected, a default number of segments is used as a fallback. This approach allows us to select the most meaningful segmentation while avoiding risks such as overfitting.

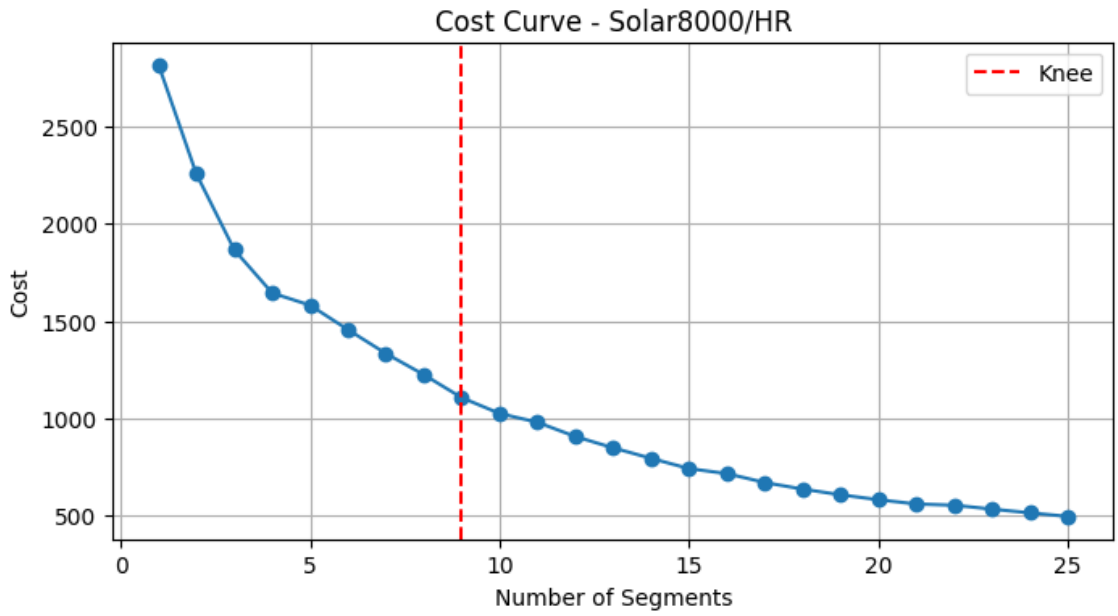

Figure 6.4: Cost Curve with Knee Locator for Bottom-Up CPD

Parameters such as the cost function RBF or the maximum number of segments were explored to optimize the detection process, this approach demonstrated that the optimal number of segments, as determined by the knee locator method, was significantly lower than the maximum number of segments (25 in this case). This highlights the importance of setting a sufficiently high maximum number of segments during the configuration of the algorithm. By doing so, the algorithm is provided with enough flexibility and space to explore a broad range of segmentations to identify the most meaningful one. If the maximum number of segments had been set too low, it would have constrained the ability of the algorithm to accurately determine the knee point and could have led to suboptimal segmentation results. The use of the knee locator was helpful in addressing this challenge, as it provided us an objective way to determine the number of segments in order to detect changes

in data.

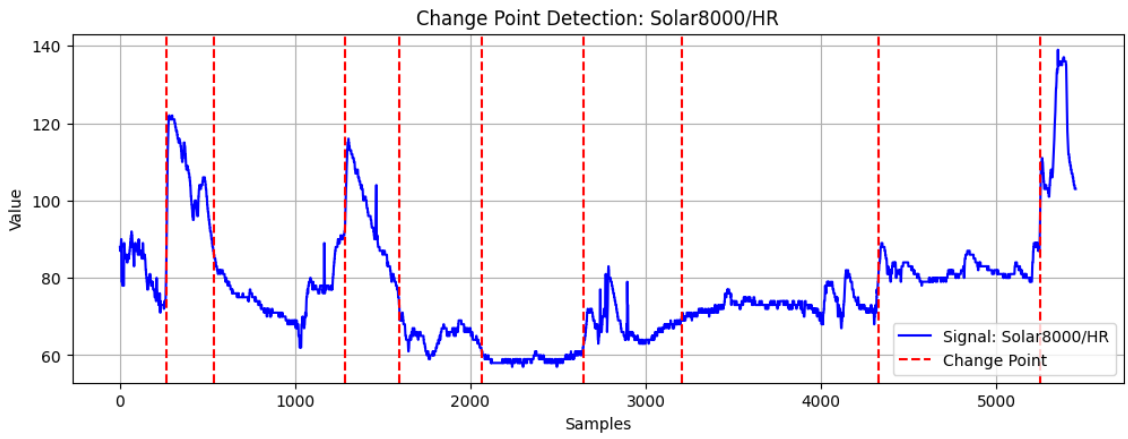

Figure 6.5: Bottom-Up CPD for Solar8000/HR

The red dashed vertical lines mark the detected change points, which are the transition points where significant changes in the data occur in the structure of the signal. These change points divide the signal into distinct segments, each representing a relatively consistent behavior. The placement of these change points aligns with noticeable changes in the tendency of the signal, as it can be appreciated in the graph, demonstrating the capacity of the algorithm to identify meaningful transitions. It is important to mention that it was necessary to remove the last change point because it was at the end of the signal, so it did not have any sense to take into account the change when the signal finished.

Overall, the Bottom-Up CPD method has effectively segmented the analyzed signal into interpretable regions. Each segment highlights identifiable trajectories, capturing distinct patterns and behaviors within the signal.

6.2.2 Other Algorithms employed

We also tried other algorithms from the library for ruptures as recommended by the people in the hospital. This library gave us flexibility, efficiency and the ability to support multiple change point detection algorithms which we will get into. We tried it into the Heart Rate's signal to identify the significant transitions. We removed the missing values from the signal to ensure the data's integrity.

PELT or Pruned Exact Linear Time Algorithm. As a model, the l2 norm was employed, as is suitable for numerical data. The parameters are min size and penalty. They define the minimum segment lenght to avoid detecting overly short segments that might represent noise, in this case, it was set to 10. Then, the other parameter

penalizes the addition of new change points, balancing sensitivy and overfitting, it was set to 10 as well.

PELT is highly effifcient and capable of processing large datasets. The implementation is quite simple, as it is built in the ruptures' library. This algorithm detected 76 change points in the HR signal.

As seen below:

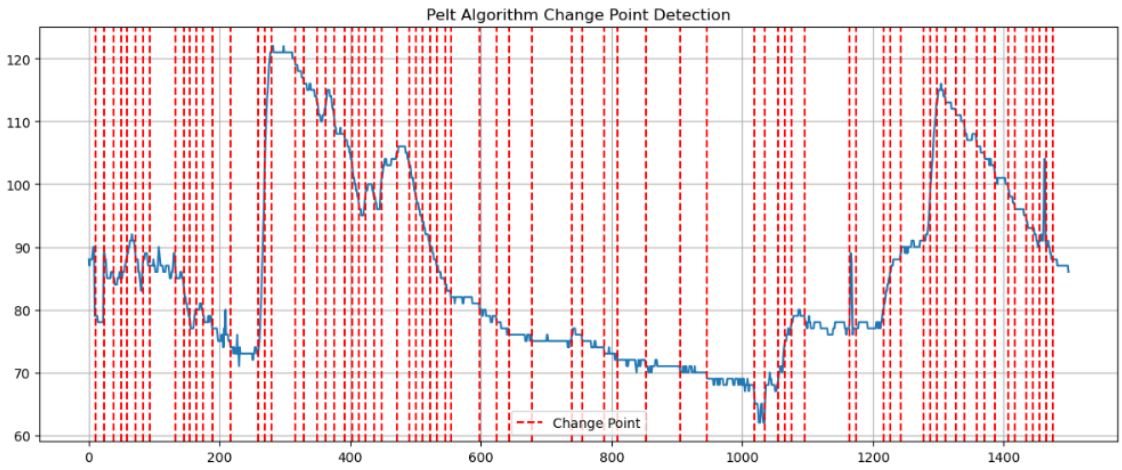

Figure 6.6: PELT

We can see that those are quite a lot of change points, which seems quite a lot of change points in this case. However, changing the minimum size to, for instance, 25, we get:

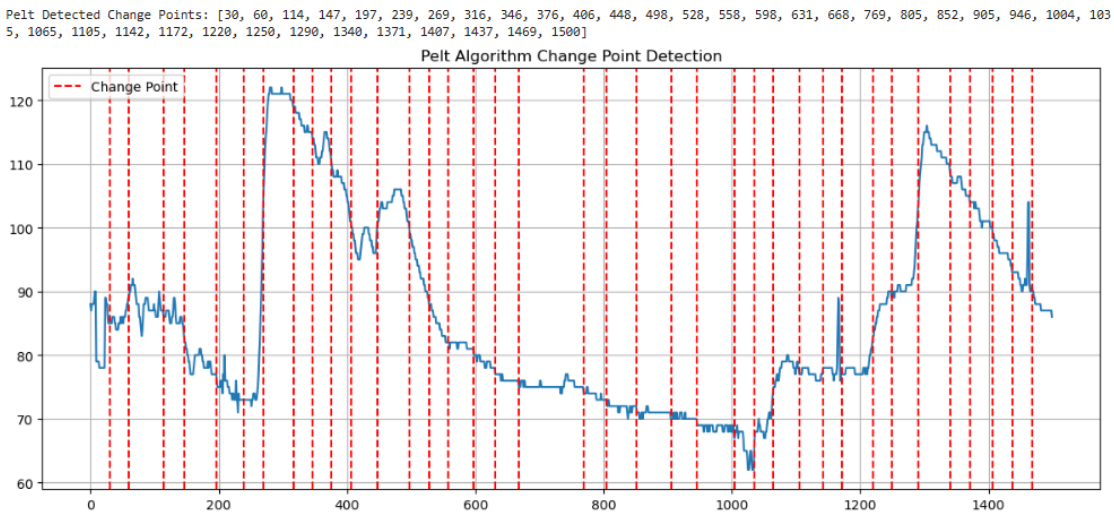

Figure 6.7: PELT2

Which seems more appropiately fitted to our signal. But it is still quite a lot of

change points.

Window-Based Change Point Detection As before, we use the l2 norm. Then as parameters, this recieves the width and the nbkps. The first referring to the size of the sliding window, for the first analysis we set it to a value of 50 and the latter refers to the number of change points to defect that was set to 10 at first, this number allows the algorithm to identify the most significant transitions. This algorithm is ideal for detecting abrupt changes in a structured data. Since our dataset is not regularly sampled and not so structured, it could not be the best for our case. It also provides a fixed-sized context for analyzing transitions within a signal, making it easier to detect localized changes. The visualization:

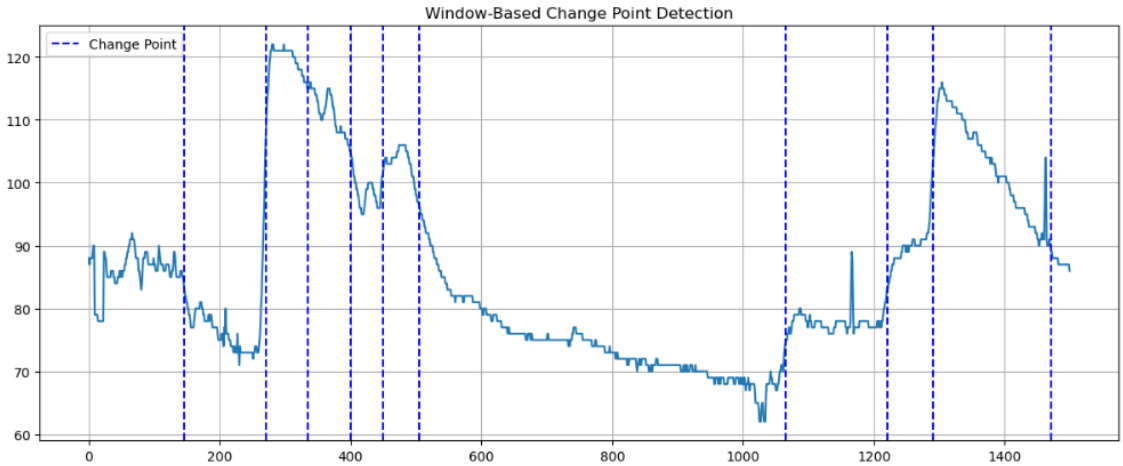

Figure 6.8: window based

Seems a bit better, since the detected change points are 11 rather than 76. However, changing a little bit the values, to a window of 55 and 20 bkps, we achieved a, seemingly, more accurate result:

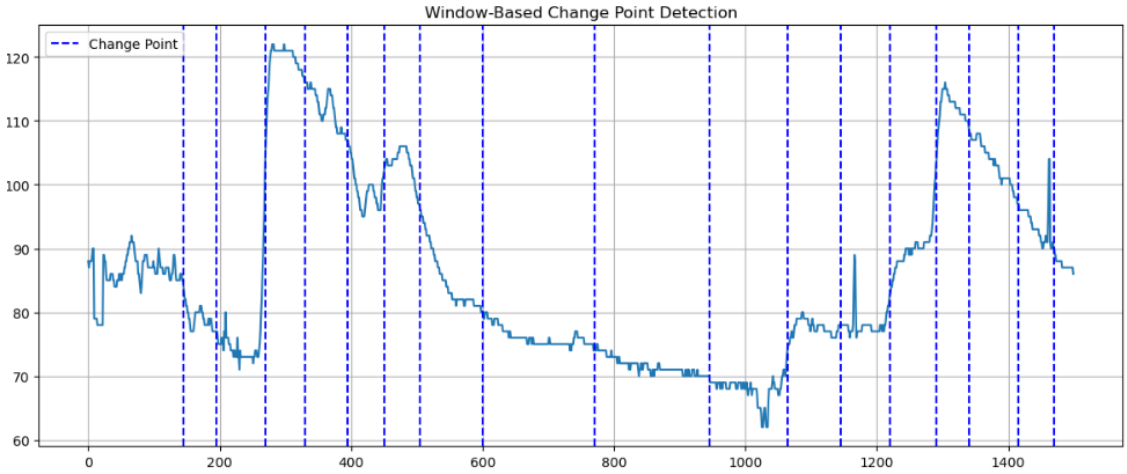

Figure 6.9: window based

Dynamic Programming (Dynp) Algorithm Again, using the l2 norm as the model (which minimizes the sum of squared errors within segments). This time we only have 1 parameter which is the number of breakpoints to detect. In our initial analysis we chose 10. Dynp guarantees a globally optimal solution by exhaustively evaluating all possible segments. It is particularly effective when the number of change points is known or can be estimated (which is not our case, but it was good to experiment with it a little bit as well). The visualization is as follows:

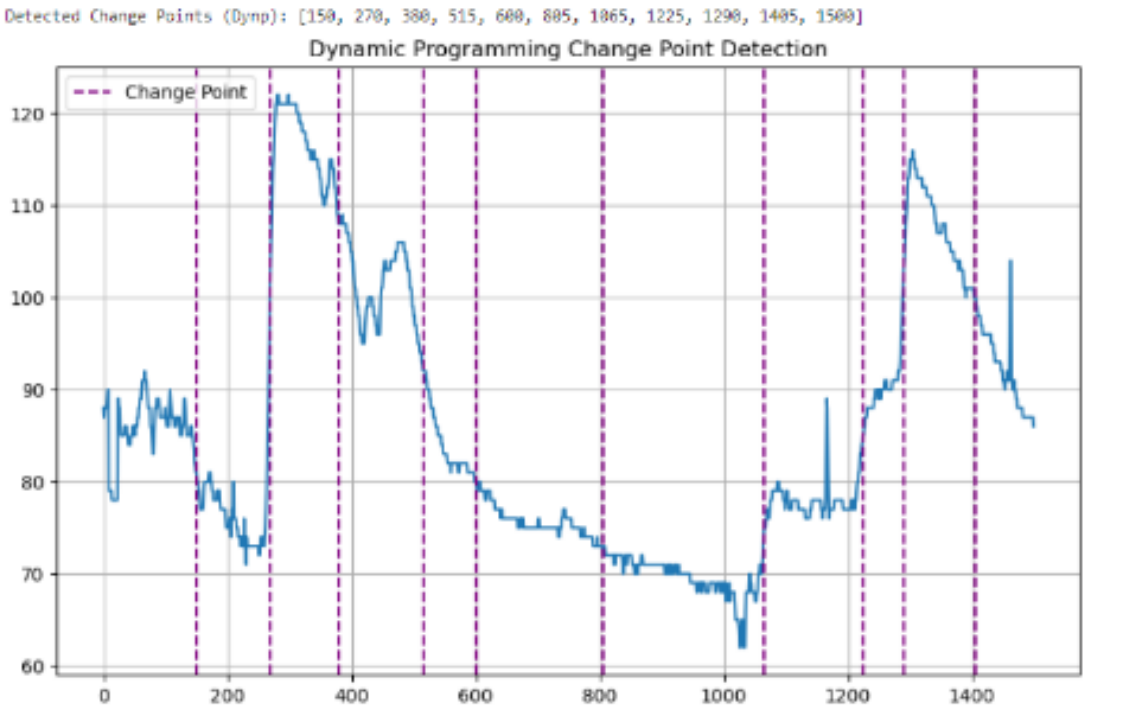

Figure 6.10: Dynamic Programming

We also changed the breakpoint number to 20 but found that for instance, the "spike" found around the value 1110, is still not taken into account.

K-means segmentation The HR signal was segmented by clustering similar points into groups using K-means, which has as a parameter the number of clusters. For this specific case, we used 3 of them. We need to reshape the signal into a 2D array, then the change points were identified as transitions between adjacent data points belonging to different clusters. We detected around 50-60 change points.

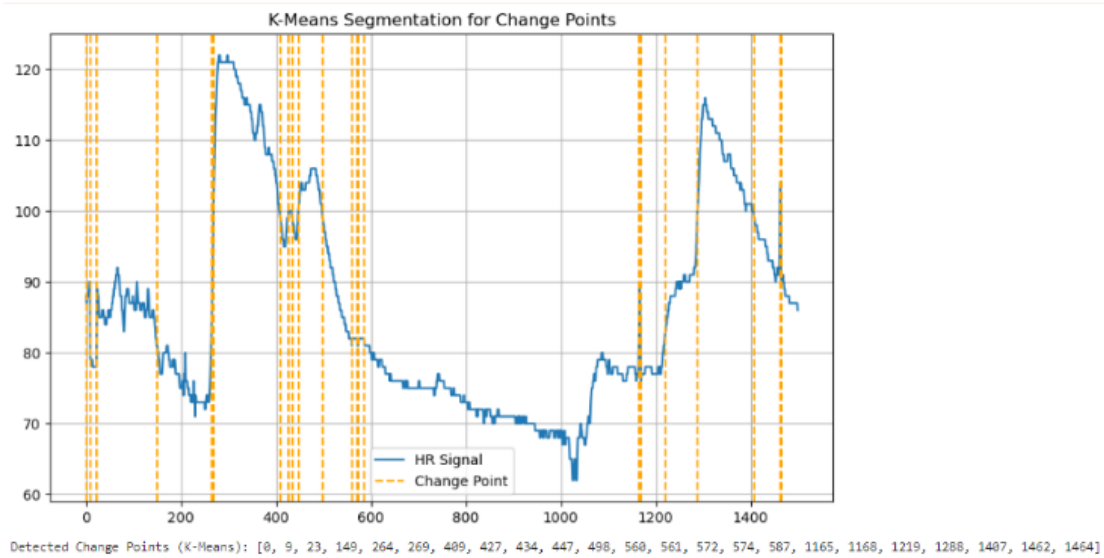

Figure 6.11: K means

Compared to other algorithms, K-means is less dependent on predefined models, making it very useful for exploratory analysis.

Threshold-Based Detection Using Derivative Lastly,

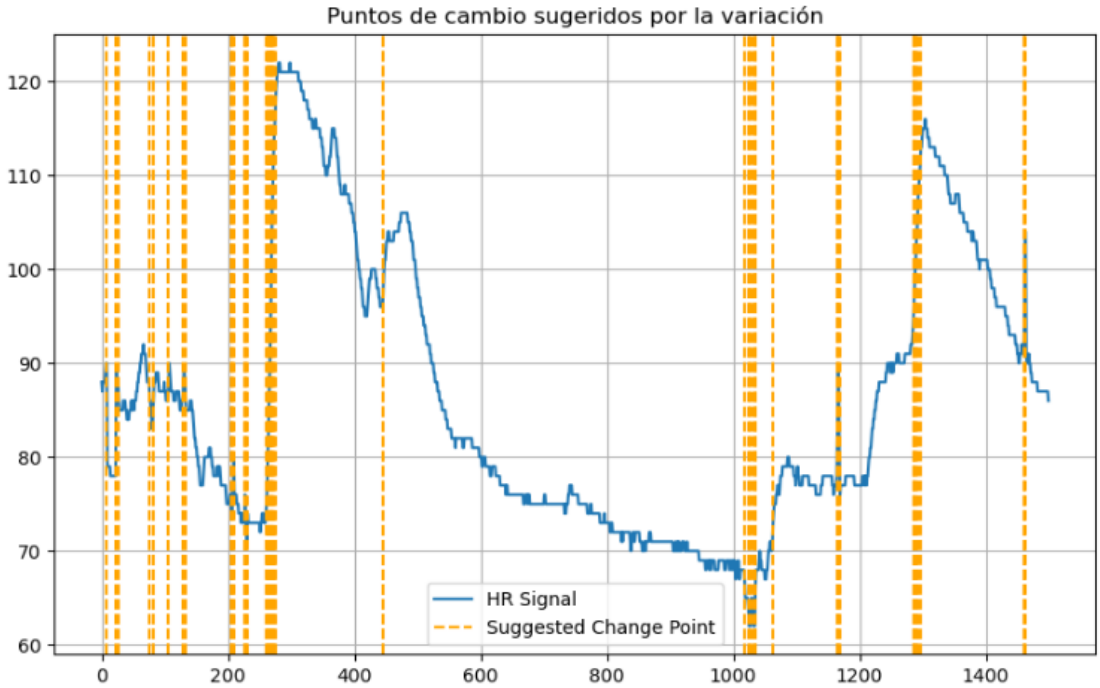

Figure 6.12: Detection using derivative

Cross- validation Afterwards, a time-series cross-validation strategy was implemented using TimeSeriesSplits with 5 folds. Unlike the standard, time-series cross-validations respects the temporal ordering of the data. Each split divides the data into a sequence of training and testing subsets, where each includes data up to a certain time point. This makes sure that the temporal integrity of the data is maintained. For each fold, the data is used to fit the change point decision algorithms. The detected change points from the algorithms are evaluated against the "true change points". Since there are not "true" change points, the detected change points from the k-means segmentation algorithm were used. Then, precision, recall and F1 score are calculated. These are the visualized algorithms used in the cross-validation:

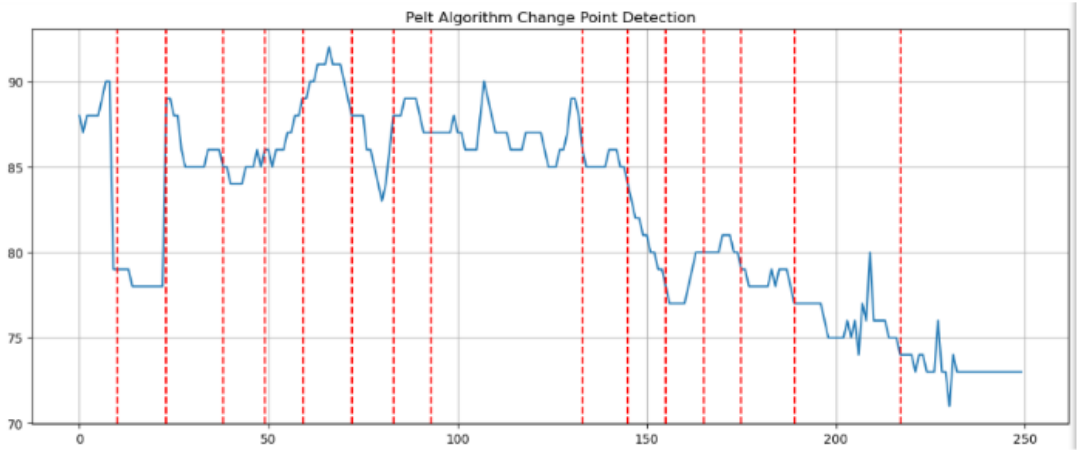

Figure 6.13: PELT detected

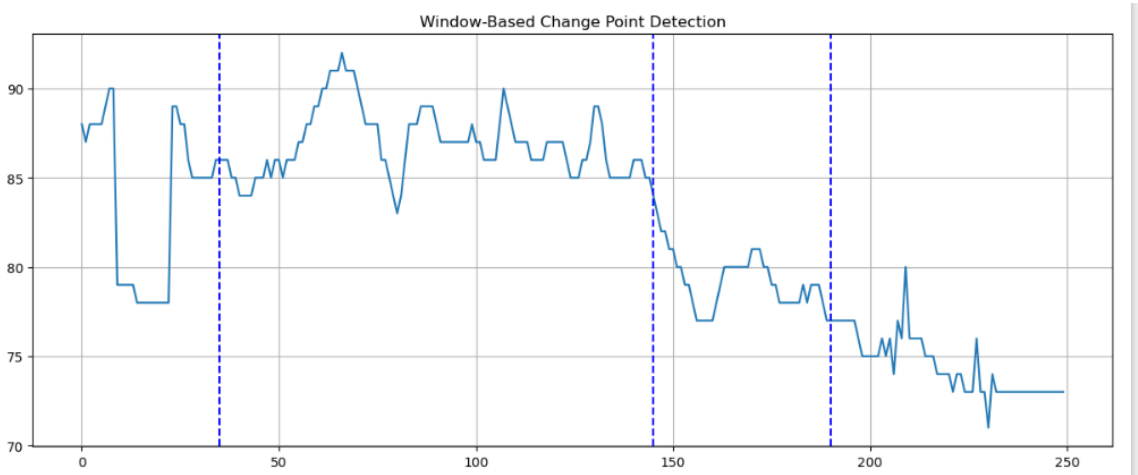

Figure 6.14: window based detected

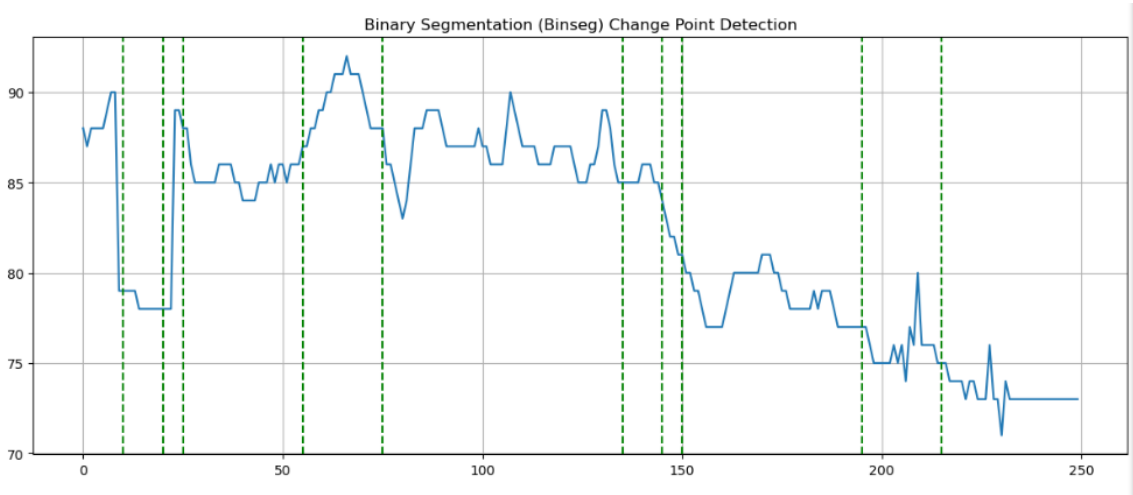

Figure 6.15: Binary Segmentation

As we can see, of course it is not perfect, but overall it seems the Binary Segmentation did the best compared to those "true change points". It seems to detect good change points. In the future, we could use some feedback to know which are the best detected points with the algorithms or ones that we know most certainly that are change points, to create a good base of "true change points".

6.3 Machine Learning Models

We also tried more advanced Machine learning models, which provide a versatile approach to change point detection, particularly for signals with complex, non-linear characteristics. In this study, we explored clustering techniques as well as LSTM-based methods, applying these techniques to heart rate data. The aim was to not only detect change points but also simulate scenarios where data is processed in both online and offline modes.

6.3.1 Clustering

Clustering methods, particularly K-means, were employed to group data points based on patterns in the heart rate signal. To prepare the data, a normalized version of the heart rate signal was computed alongside the original magnitude. This allowed for the creation of point matrices, where each row represented a specific data point and its historical context. Two key parameters influenced the creation of these matrices: the *window size*, defining how many previous points were considered, and the *step size*, determining the interval at which rows were added. For example, with a window size of 100 and a step size of 1, the matrix row for point 100 included its

normalized value, the values of the preceding 99 points, and its original magnitude.

An example of such a matrix is shown in Table 6.1. Here, columns *signal1* through *signal100* represent the normalized values of the current and previous 99 points, while the column *magnitude* contains the original, unnormalized value of the current point (in this case, point 100).

Table 6.1: Example of a matrix row for clustering with a window size of 100.

signal1	**signal2**	**...**	**signal100**	**magnitude**
0.12	0.15	...	0.20	85

This structure ensures that each data point captures both its normalized historical context and its actual magnitude, allowing for effective clustering based on both temporal patterns and original signal characteristics.

Various configurations of window and step sizes were tested. Larger window sizes did not show a significant difference in clustering results. However, increasing the step size to values such as 5 introduced more gaps in the data, which led to less cohesive patterns. Once clusters were formed, the K-means cluster centers were extracted and reused for prediction on new datasets, simulating an online detection scenario.

Online vs Offline Simulation To simulate an online approach, the K-means cluster centers derived from the model trained on one dataset were applied to a new dataset for predictions. A key challenge in this process was normalization, as future values of the new dataset were not available for scaling. To address this, the mean and standard deviation from the training dataset were used to normalize the data in the new dataset, ensuring consistency in feature scaling.

A new prediction matrix was constructed using these normalized values, allowing the trained model to cluster the incoming data points in real time. The results demonstrated that the clustering patterns from the training dataset aligned well with the predictions on the new dataset, even when the signal exhibited stable magnitudes in certain sections. This suggests that clustering methods can effectively capture change points, even under varying signal dynamics.

For the offline approach, the entire dataset was processed at once, enabling a global perspective on clustering patterns. While this method provided similar results, it lacked the adaptability required for real-time detection scenarios.

Figures 6.16 and 6.17 illustrate the clustering applied to the training dataset, showing both the raw signal and the clusters formed. Figures 6.18 and 6.22 present the results of applying the trained model to a new dataset, highlighting the online simulation of change point detection.

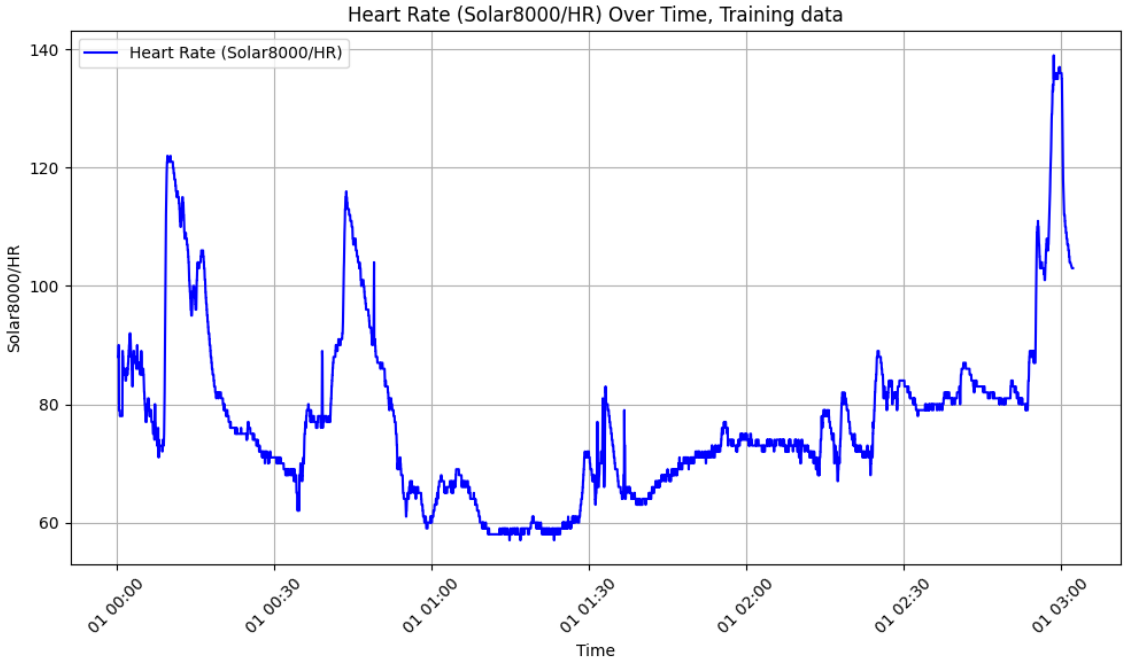

Figure 6.16: Heart rate signal from the training dataset.

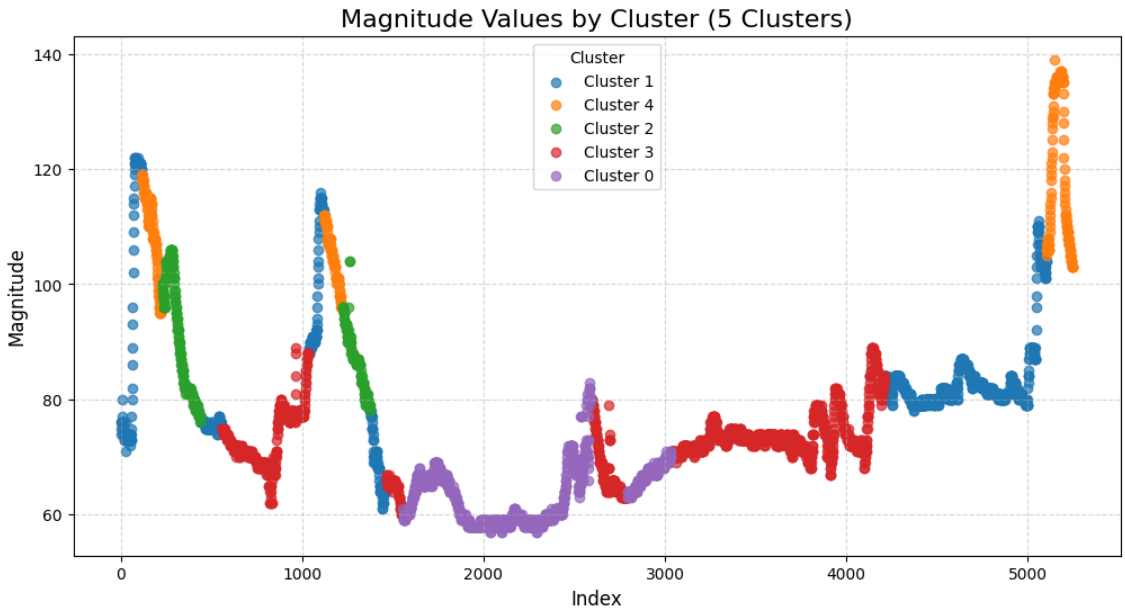

Figure 6.17: Magnitude values grouped by predicted cluster for the training dataset.

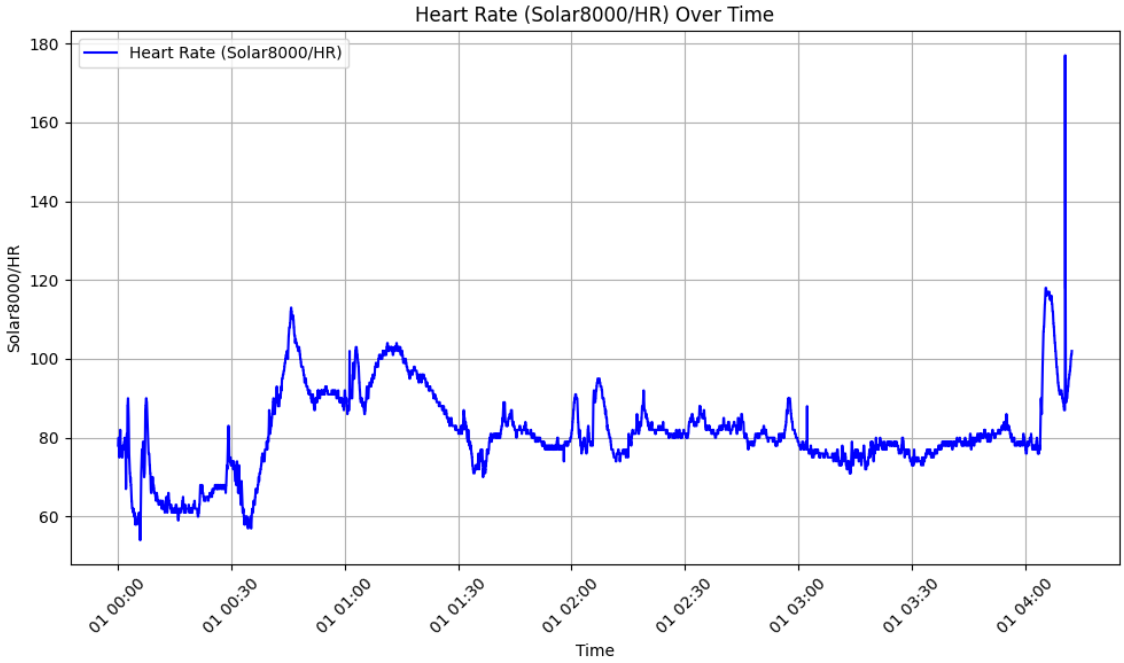

Figure 6.18: Heart rate signal from the prediction dataset.

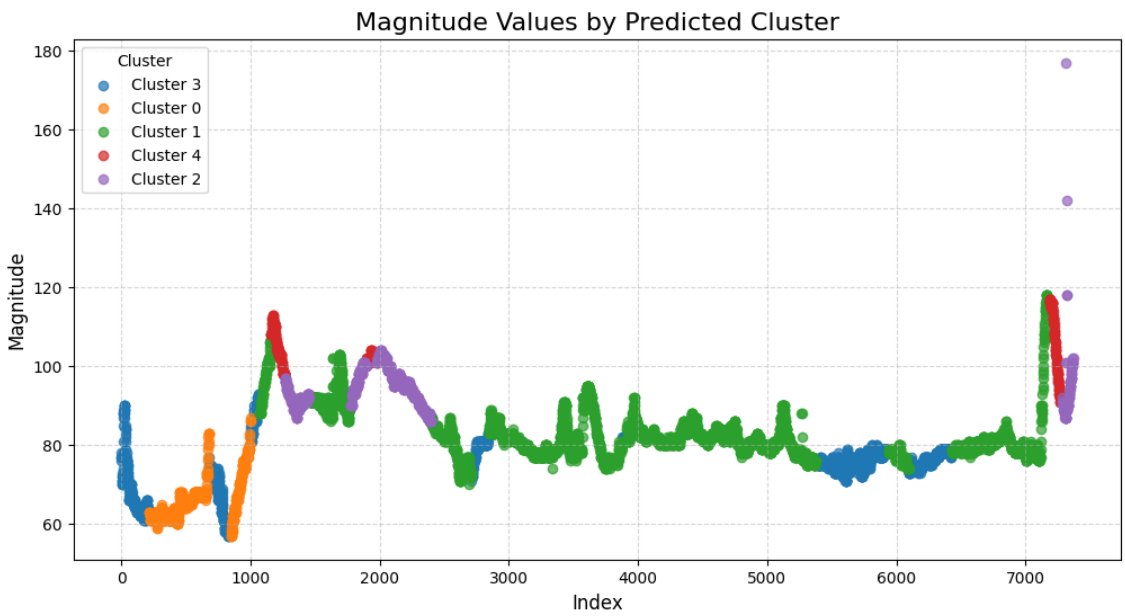

Figure 6.19: Magnitude values grouped by predicted cluster for the prediction dataset.

6.3.2 LSTM-based methods

Change Point Detection using LSTM models can be approached in two primary ways: the predictive modeling approach and the reconstruction-based approach using autoencoders. In the predictive modeling approach, an LSTM model is trained to predict future values of the time series based on past observations. Significant deviations between the predicted and actual values are analyzed to identify change points where the statistical properties of the data shift. In the reconstruction-based approach, an LSTM autoencoder is trained to learn the typical patterns of the time series. Reconstruction errors are then examined to detect points where the data diverge from the learned patterns, indicating potential changes. In this project, we implemented and evaluated both approaches using the Solar8000/HR signal, a metric present in the hospital's dataset too, ensuring that our findings can be easily integrated into their system. The main advantage of using the same signal for both methods is that it allows us to directly compare their performance and draw consistent conclusions.

LSTM autoencoder approach

To implement the reconstruction-based approach for Change Point Detection, we utilized an LSTM autoencoder model trained on the Solar8000/HR signal. The model was designed to learn the inherent patterns of the signal by compressing it into a latent space and reconstructing it back to its original form. Significant reconstruction errors were then analyzed to identify points where the signal deviated from the learned patterns, suggesting potential changes in the underlying data distribution. The LSTM autoencoder was configured with a single LSTM layer consisting of 50 units, followed by a Dropout layer with a 0.2 rate to mitigate overfitting, and a Dense layer for final reconstruction. The model was trained for 2 epochs using the Adam optimizer and Mean Squared Error (MSE) as the loss function.

We experimented with various hyperparameter combinations, including different numbers of LSTM units (e.g. 50, 100, and 200), dropout rates (0.2 to 0.5), and epochs (2 to 10). Among these, the configuration with 50 units and 2 epochs provided a good balance between reconstruction accuracy and computational efficiency. This combination captured the essential dynamics of the Solar8000/HR signal without overfitting, making it well-suited for real-time applications in a clinical setting. By experimenting with different hyperparameters we saw that the number of detected change points changed, and also their position.

To create the sequences from the original signal for the LSTM model, we tested different combinations of sequence length and overlap. Ultimately, we chose a sequence length of 100 samples and compared two overlap configurations: 99 samples and 20 samples. The configuration with 99 samples of overlap resulted in a higher

number of sequences, preserving more temporal information but also increasing computational complexity. In contrast, the configuration with 20 samples of overlap provided fewer sequences, offering a better trade-off between computational efficiency and detection performance. We found that smaller sequence lengths or higher overlaps led to more detected change points, some of which were true positives. Based on these observations, we concluded that the combination of 100 sequence length and 20 overlap was the most effective for this application.

To find the change points, reconstruction errors were visualized and analyzed. The threshold for detecting these points was determined dynamically based on the mean reconstruction error plus a factor of 1.5 times the standard deviation. Detected change points were marked as red dots on the reconstructed signal plots to provide a clear and intuitive representation.

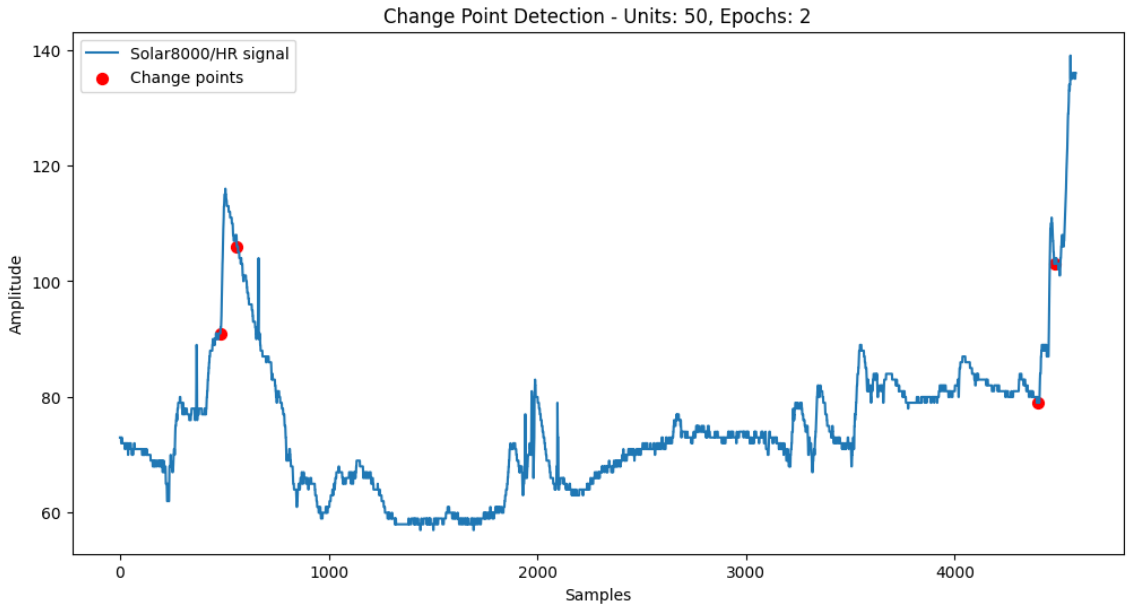

Figure 6.20: Change points detected using LSTM as autoencoder with sequeces of length 100 and overlap 20.

LSTM predictive modeling approach

For the predictive modeling approach, we implemented an LSTM model trained on the Solar8000/HR signal to predict the next value in the time series based on a sequence of previous values. The model leverages the temporal dependencies in the data to make accurate predictions, and significant deviations between the predicted and actual values were analyzed to detect change points where the signal's behavior shifted. The LSTM model was configured with a single LSTM layer containing 50 units, followed by a dropout layer with a rate of 0.2 to avoid overfitting and a dense layer to output the predicted value. The model was trained for 20 epochs using the

Adam optimizer with a learning rate of 0.0003 and a mean squared error (MSE) as a loss function.

Again, before arriving at this configuration, we experimented with multiple combinations of hyperparameters and loss functions to identify the most suitable setup for our predictive modeling approach. We tested different numbers of LSTM units (e.g. 25, 50, 100), epochs (ranging from 10 to 50), and dropout rates (from 0.1 to 0.5). Additionally, we evaluated various loss functions, including the mean absolute error (MAE) or Huber loss along with the mean squared error (MSE). While MAE demonstrated robustness to outliers and Huber Loss offered a compromise between sensitivity and stability, we ultimately selected MSE due to its sensitivity to larger errors, making it particularly effective for capturing significant deviations in the Solar8000/HR signal.

In terms of optimization algorithms, we considered alternatives such as SGD, but Adam proved to be the most effective due to its adaptive learning rate, which facilitated stable and efficient training. The learning rate of 0.0003 was determined through experimentation, balancing between convergence speed and avoiding overfitting. This final configuration (50 LSTM units, dropout of 0.2, Adam optimizer with 0.0003 learning rate, and MSE loss) provided the best trade-off between model complexity, performance, and computational efficiency. These choices ensured the model's ability to generalize well to the signal dynamics and reliably detect significant change points.

In the data preparation stage, we divided the signal into sequences of 100 with an overlap of 99 samples to ensure that temporal structure was preserved. With this setup, many such overlapping sequences were prepared that provided enough training data to capture even complex patterns in the signal. This is further supported by the evolution of the loss function during training, which shows its capability of learning and generalizing the dynamics of the signal effectively.

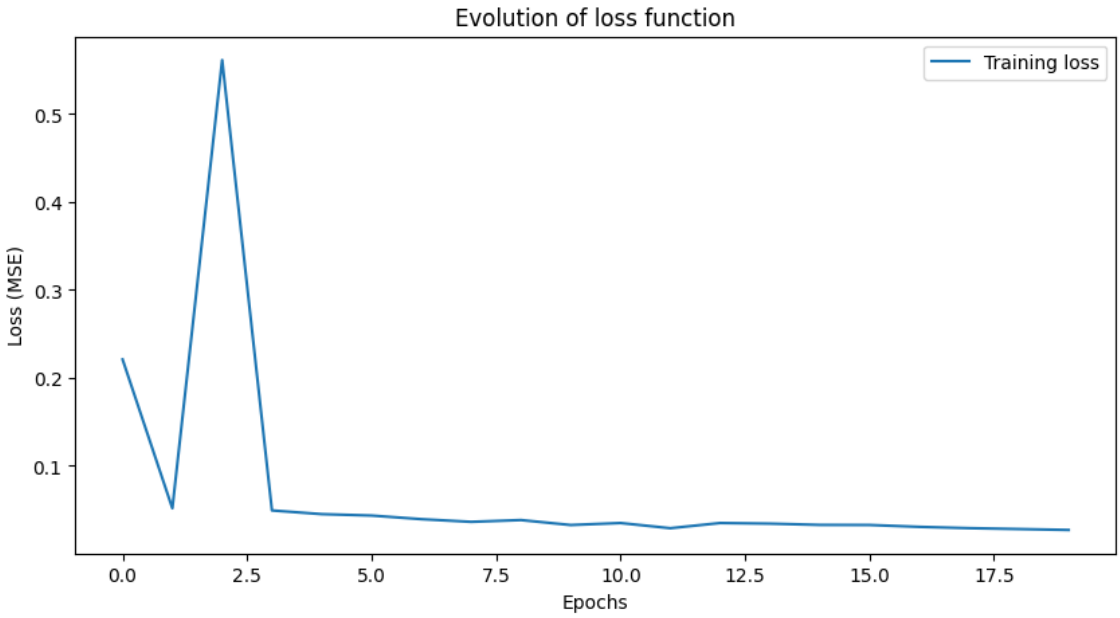

Figure 6.21: Evolution of the loss function during training for the LSTM predictive model.

Once trained, the LSTM model was used to predict the next value for each sequence. These predictions were then reconstructed into a signal to compare with the original data. Differences between the predicted and actual values were calculated, and a threshold was dynamically defined as the mean difference plus 1.5 times the standard deviation. Initial change points were identified where the differences exceeded this threshold.

To refine the detection process, a simplification algorithm was applied to the raw change points. This algorithm consolidated consecutive detections into a single point, using a tolerance of 50 samples to determine which points belonged to the same cluster. By reducing redundant detections, this step ensured that only meaningful change points were retained for further analysis.

The following plot shows the original signal, the reconstructed signal from the predictions, and the simplified change points marked in red. These visualizations provide a clearer and more intuitive understanding of how the model identifies significant deviations and transitions in the signal

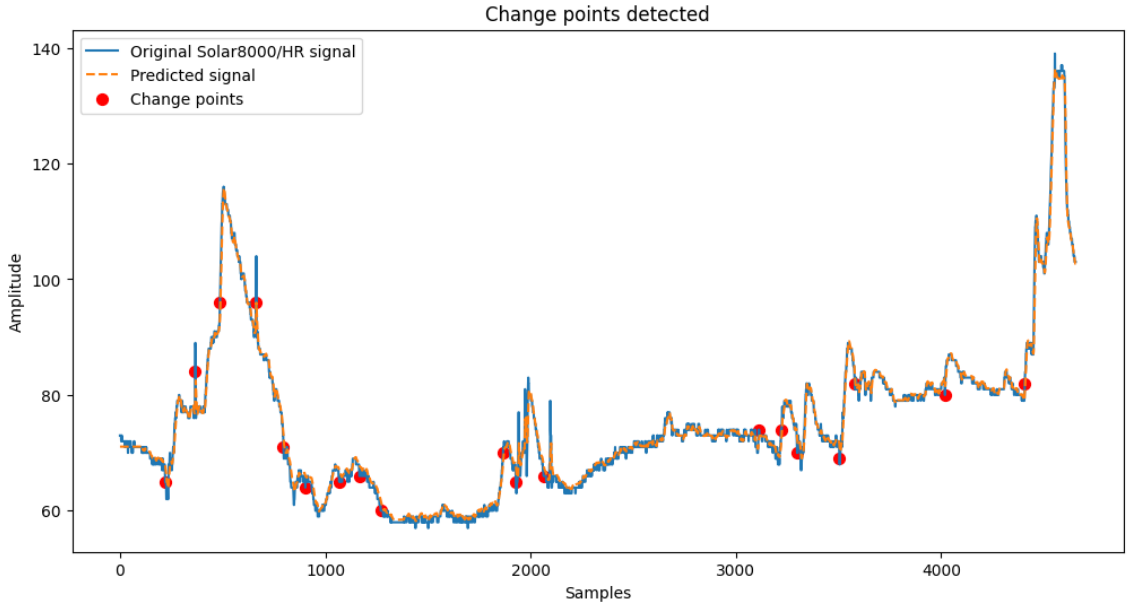

Figure 6.22: Change points detected using the LSTM predictive approach on the Solar8000/HR signal.

Through experimentation, we observed that smaller overlaps or longer sequences resulted in fewer detected change points, as the model prioritized broader trends over localized fluctuations. Conversely, shorter sequences or higher overlaps produced more frequent detections, some of which were less significant. The chosen configuration of 100 sequence length and 99 overlap struck a balance between precision and sensitivity, making it suitable for identifying clinically relevant change points.

Comparison of the two approaches

The results of the two LSTM-based approaches—reconstruction-based (autoencoder) and predictive modeling—reveal notable differences in the detected change points. The reconstruction-based method, with a sequence overlap of 20, identified fewer but more localized change points, likely due to the shorter overlap capturing sharp transitions in the signal. This method's strength lies in its sensitivity to sudden deviations from the learned patterns, making it well-suited for detecting abrupt or isolated changes. On the other hand, the predictive modeling approach, with a higher overlap of 99, produced a denser set of change points that are more distributed along the signal. This approach appears to prioritize broader trends and gradual shifts, capturing a wider range of transitions in the data.

One key observation is that the predictive modeling approach tends to detect more transitions, some of which might be false positives due to its sensitivity to

minor deviations. Conversely, the reconstruction-based approach, with fewer change points, may focus more on significant and clinically relevant anomalies. However, in our non expert opinion, the change points detected by the predictive approach seem better, even if they are more.

It would be great to count with the help of a medical expert that could help us determine which change points are better, something that would be very useful to choose an approach. However, right now, without that expert advice, we would conclude that the reconstruction-based method might be more aligned with the goal of detecting critical shifts while avoiding unnecessary noise, while the predictive modeling approach offers valuable insights into broader trends, which could complement the reconstruction-based method in a multi-layered analysis.

Another option would be to combine both approaches: by combining their strengths, we could create a more robust framework for change point detection that balances precision and sensitivity, but this would just be an idea for future work.

Chapter 7

Challenges and conclusions

7.1 Challenges and Limitations

One of the primary and most important limitations we faced was the inability of the hospital to provide us with the dataset containing patient data due to privacy concerns. Despite their initial intention to share the data to develop a project in collaboration with their institution, the legal restrictions did not make it possible to use their data for our models. As a result, we had to adapt and carry out the project using the VitalDB database, as we explained before. Although this issue was beyond our control, we adjusted our approach to work with this alternative dataset.

During the development of the project, we realized that clinical data often contains a significant amount of gaps in the dataset. This is mainly due to the fact that the sensors do not always collect data from patients, as they can become disconnected, powered off, or otherwise inactive. This presented a considerable challenge, as we had to adapt to the context of our dataset by focusing exclusively on the periods where the sensors were actively obtaining data. Once detected, we just needed to focus on the active time frames.

A further complication arises from the fact that the dataset we are working with (like the data from the hospital) is unlabeled, meaning that there are no explicit labels indicating if a point was or not an anomaly, or a meaningful change point. This has been the most significant limitation during our project, as the lack of labeled data has prevented us from verifying the true performance of our models.

To address this challenge, we proposed organizing a meeting with medical professionals of the Hospital Niño Jesús to get their insights on where anomalies might be present in our signals, or whether the detected change points were relevant. This information could have allowed us to apply a form of semi-supervised learning, combining their expertise with our models to improve anomaly detection. However, because of time constraints and scheduling difficulties, it was not possible to arrange

this meeting before the deadline for the submission of the final report.

What makes the previously mentioned challenge harder to deal with is our limited medical knowledge. Although we were able to make some educated guesses about where anomalies might be based on common sense, this remained a significant limitation and a not reliable approach. The lack of domain expertise drastically limited our ability to confidently interpret the data and refine our models, further remarking the importance of collaboration with medical professionals, or labeling the data, for these types of project.

7.2 Final Conclusions

The objective of the project was to detect anomalies in clinical time-series data and identify turning or change points in the data, as well as their strengths and weaknesses. Several techniques were deployed and tested, including One-Class SVM, Isolation Forest, LSTM models, and Autoencoders (Dense, CNN-based, and Transformer-based) for detecting anomalies. Among these, the CNN and especially the Transformer autoencoders were promising, showing potential for further development. When fine-tuned, these models could enable the detection of patterns in high-dimensional datasets. For change point detection, CUSUM, kernel methods, and machine learning methods, particularly LSTM-based models in combination with clustering methods, proved effective. LSTMs are especially useful for predictive modeling, particularly in identifying slow and rapid changes in signals such as heart rate, with two distinct approaches enabling different focuses, as explained earlier.

A potential area for future work would be a detailed study of the datasets used to train the Transformer autoencoder for anomaly detection and the LSTM predictive model for change point detection. This would involve utilizing an experts knowledge to label the dataset, allowing for the creation of more accurate models, since there would be some ground truth to compare it to.

It would also be highly beneficial to have an expert assist in determining more precisely which of the created models work best based on the obtained results. While our evaluations were based on what seemed appropriate from a data-driven analytical perspective, an expert would have a much deeper understanding of the clinical context. Their feedback would not only validate our results but could also provide surprising insights we might not have considered, ultimately enhancing the effectiveness and applicability of our models.

The project successfully explored a wide range of anomaly detection and change point detection methods, but further work is needed to turn these into a robust real-time clinical tool. In particular, detecting false positives remains a challenge, and it would be important to work with medical professionals to determine acceptable

thresholds for triggering alarms. Additionally, the computational cost of methods such as Transformers and LSTMs must be balanced with the need for timely results in an ICU environment.

From an ethical perspective, any deployment of such models must be done in full compliance with strict regulations concerning patient data privacy and security. Collaboration with hospital IT departments and subject-matter experts will be crucial to ensure both the validity of the models' outputs and alignment with patient care workflows. Furthermore, implementing our models in the hospital system and even fine-tuning them with the hospital dataset, if considered, would be vital. Eventually, the outputs of these anomaly and change point detection algorithms could be integrated into an overall predictive monitoring system that incorporates additional data on patients and the clinical context, delivering comprehensive decision support to medical teams.

Bibliography

[1] S. Agrawal and J. Agrawal, "Survey on anomaly detection using data mining techniques," *Procedia Computer Science*, vol. 60, pp. 708–713, 2015, https://doi.org/10.1016/j.procs.2015.08.220.

[2] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," arXiv pre-print 1901.03407 cs.LG, Jan. 2019, https://doi.org/10.48550/arXiv.1901.03407.

[3] M. Braei and S. Wagner, "Anomaly detection in univariate time-series: A survey on the state-of-the-art," *ArXiv*, vol. abs/2004.00433, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:214743362

[4] G. E. Hinton and R. S. Zemel, "Autoencoders, minimum description length and Helmholtz free energy," in *Proceedings of the 6th International Conference on Neural Information Processing Systems*, ser. NIPS'93. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1993, pp. 3–10. [Online]. Available: https://dl.acm.org/doi/abs/10.5555/2987189.2987190

[5] M. Hejazi and Y. Singh, "One-class support vector machines approach to anomaly detection," *Applied Artificial Intelligence*, vol. 27, pp. 351–366, 05 2013. [Online]. Available: https://doi.org/10.1080/08839514.2013.785791

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sep. 1995. [Online]. Available: https://doi.org/10.1023/A:1022627411411

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[9] S. Haykin, *Neural networks: A comprehensive foundation*. Englewoods Cliffs, NJ, USA: Macmillan Publishing Company, 1994.

[10] B. Widrow, D. E. Rumelhart, and M. A. Lehr, "Neural networks: Applications in industry, business and science," *Communications of the ACM*, vol. 37, no. 3, pp. 93–105, Mar. 1994. [Online]. Available: https://doi.org/10.1145/175247.175257

[11] Y. Bengio, "Learning deep architectures for AI," *Foundations and Trends in Machine Learning*, vol. 2, pp. 1–127, 2009. [Online]. Available: https://doi.org/10.1561/2200000006

[12] B. Schölkopf, J. C. Platt, J. C. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001. [Online]. Available: https://doi.org/10.1162/089976601750264965

[13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD'96. AAAI Press, 1996, pp. 226–231. [Online]. Available: https://dl.acm.org/doi/10.5555/3001460.3001507

[14] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, pp. 89–94, 2015. [Online]. Available: https://arxiv.org/abs/1506.03390

[15] T. Fernando, H. Gammulle, S. Denman, S. Sridharan, and C. Fookes, "Deep learning for medical anomaly detection – A survey," *ACM Computing Surveys (CSUR)*, vol. 54, no. 7, pp. 141:1–141:37, Jul. 2021. [Online]. Available: https://doi.org/10.1145/3464423

[16] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Computing Surveys*, vol. 54, no. 2, Mar. 2021. [Online]. Available: https://doi.org/10.1145/3439950

[17] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Computing Surveys*, vol. 54, no. 3, Apr. 2021. [Online]. Available: https://doi.org/10.1145/3444690

[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[19] Y. Yu, S. Zhu, S. Li, and D. Wan, "Time series outlier detection based on sliding window prediction," *Mathematical Problems in Engineering*, vol. 2014, October 2014. [Online]. Available: https://doi.org/10.1155/2014/879736

[20] L. Kulanuwat, C. Chantrapornchai, M. Maleewong, P. Wongchaisuwat, S. Wimala, K. Sarinnapakorn, and S. Boonya-aroonnet, "Anomaly detection using a sliding window technique and data imputation with machine learning for hydrological time series," *Water*, vol. 13, no. 13, 2021. [Online]. Available: https://www.mdpi.com/2073-4441/13/13/1862

[21] L. Ma, X. Gu, and B. Wang, "Correction of outliers in temperature time series based on sliding window prediction in meteorological sensor network," *Information*, vol. 8, no. 2, 2017. [Online]. Available: https://www.mdpi.com/2078-2489/8/2/60

[22] A. Siva Kumar, S. Raja, N. Pritha, H. Raviraj, R. Babitha Lincy, and J. Jency Rubia, "An adaptive transformer model for anomaly detection in wireless sensor networks in real-time," *Measurement: Sensors*, vol. 25, p. 100625, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2665917422002598

[23] J. Meng, L. Liu, Y. Liu, and N. Wang, "WD: A sliding window based time series compression algorithm," in *International Conference on Machine Learning, Big Data and Business Intelligence*, Dec. 2023, pp. 148–152. [Online]. Available: https://doi.org/10.1109/MLBDBI60823.2023.10482339

[24] L. Guo and Y. Wei, "A time series segment finding motifs based on sliding window algorithm," in *2024 IEEE International Conference on Industrial Technology (ICIT)*, 2024, pp. 1–6. [Online]. Available: https://doi.org/10.1109/ICIT58233.2024.10540977

[25] C. Truong, L. Oudre, and N. Vayatis, "Selective review of offline change point detection methods," *Signal Processing*, vol. 167, p. 107299, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0165168419303494

[26] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[27] H. Chernoff, "On the distribution of the likelihood ratio," *The Annals of Mathematical Statistics*, vol. 25, no. 3, pp. 573–578, 1954.

[28] Z. Harchaoui and O. Cappe, "Kernel change-point analysis," *Advances in Neural Information Processing Systems*, vol. 19, pp. 609–616, 2007. [Online]. Available: https://papers.nips.cc/paper/2007/file/af12632a28001a4e62f9acb5af29c804-Paper.pdf

[29] Z. Harchaoui, O. Cappe, and E. Moulines, "Regularized kernel-based change point detection," *Journal of Machine Learning Research*, vol. 11,

pp. 1517–1540, 2009. [Online]. Available: http://jmlr.org/papers/volume11/harchaoui10a/harchaoui10a.pdf

[30] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 2114–2124, 2021.

[31] H. C. Lee, Y. Park, S. B. Yoon, S. M. Yang, D. Park, and C. W. Jung, "VitalDB dataset," https://vitaldb.net/dataset/, 2022, accessed: 2024-10-18.

[32] ——, "VitalDB, a high-fidelity multi-parameter vital signs database in surgical patients," *Scientific Data*, vol. 9, no. 279, pp. 1–9, 2022. [Online]. Available: https://doi.org/10.1038/s41597-022-01411-5